

SAPIENZA
UNIVERSITÀ DI ROMA

"SAPIENZA" UNIVERSITY OF ROME

FACULTY OF INFORMATION ENGINEERING,
COMPUTER SCIENCE AND STATISTICS

DEPARTMENT OF COMPUTER, CONTROL
AND MANAGEMENT ENGINEERING

MASTER'S DEGREE IN ARTIFICIAL INTELLIGENCE AND ROBOTICS

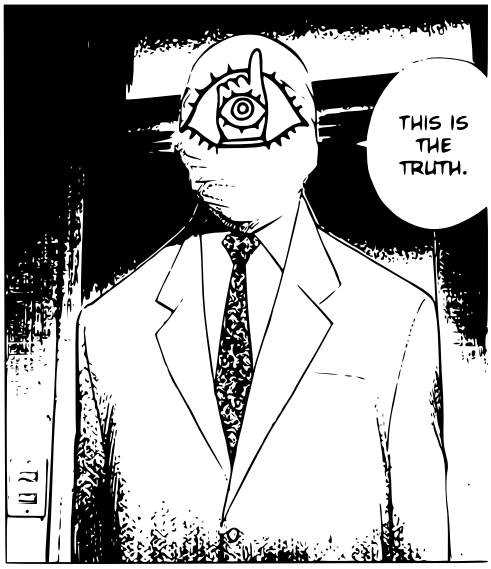
Elective in Robotics

Lecture notes

Author
Marco Casu
https://github.com/CasuFrost

October 14, 2026

This document summarizes and presents the topics for the Elective in Roboticscourse for the Master's degree in Artificial Intelligence and Robotics at Sapienza University of Rome. The document is free for any use. If the reader notices any typos, they are kindly requested to report them to the author.



CONTENTS

I Modeling and Control of Multi-Rotor UAVs **3**

1 Introduction **4**
1.1 Quadrotor Modeling 5
1.2 Quadrotor Control 9
1.2.1 Linear Control 10
1.2.2 Basic Control and Planning Problems 11

2 Control Methods **14**
2.1 Backstepping 14
2.1.1 Backstepping for Height Control 16
2.1.2 PD Compared With Backstepping 17
2.1.3 Integral Backstepping 17
2.2 Dynamic Feedback Linearization 18
2.2.1 Static Linearization 19
2.2.2 Input-Output Linearization 20
2.2.3 Linearization of the Quadrotor Model 21
2.2.4 Considering the Complete Model 24
2.2.5 Flatness 26
2.3 Geometric Control on $SO(3)$ 27

Part I

Modeling and Control of Multi-Rotor UAVs

CHAPTER

1

INTRODUCTION

UAV stands for **Unmanned Aerial Vehicle**, also known by other names depending on the domain, like *RPVs (Remotely Piloted Vehicles)* in the military domain, or *Drones* in common parlance. We will use all three names interchangeably. The methods presented are valid also for remotely operated or autonomous vehicles (at different levels of autonomy).

The term UAVs refers to a broad category of vehicles; in general, any aerial vehicle without a pilot on board is a UAV. We will examine UAVs composed of multiple rotors, in particular **quadrotors**.



Figure 1.1: Design-based classification.

Quadrotors are used in various situations:

- inspection, maintenance,
- communication networks,
- search and rescue operations,
- aerial transportation, manipulation.

These activities require a vertical, stationary and slow flight. A quadrotor is characterized by high maneuverability, can remain suspended in the air without moving (hovering), and can perform a *vertical take-off and landing (VTOL)*, unlike other vehicles such as airplanes, which require a horizontal movement in order to take off and land.

1.1 Quadrotor Modeling

We call VTOLs the vehicles that can perform a vertical take-off and landing; these are modeled with the North-East-Down (NED) reference frame:

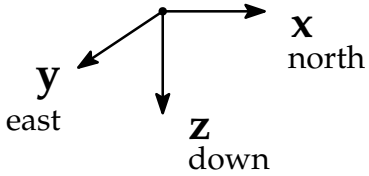


Figure 1.2: The NED reference frame.

This frame, often called SR_i, has the $\mathbf{z}$ axis pointing down to the center of the Earth. According to the dynamics of rigid bodies given by the Newton-Euler formalism, the translational dynamics of a body in the SR_i frame is given by

$$\sum_j \mathbf{F}_j = m\dot{\mathbf{V}} \tag{1.1}$$

where $\mathbf{F}_j$ are the external forces applied to the center of mass, and $\dot{\mathbf{V}}$ is the acceleration of the center of mass expressed in SR_i. The rotational dynamics is given by

$$\sum_j \mathbf{M}_j = J\dot{\mathbf{\Omega}} + \mathbf{\Omega} \times J\mathbf{\Omega} \tag{1.2}$$

where $\mathbf{M}_j$ are the external moments acting on the center of mass, $\mathbf{\Omega}$ is the angular velocity of the body expressed in the body frame, denoted SR_b, and J is the inertia tensor, also expressed in SR_b.

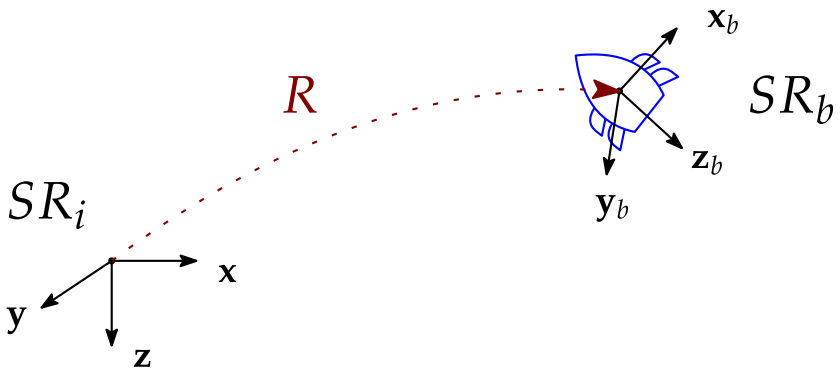


Figure 1.3: SR_b is the reference frame attached to the center of mass of the vehicle, R is the rotation matrix between the two frames.

A more explicit expression of the forces in Equation (1.1) is the following:

$$m\dot{\mathbf{V}} = -TR\mathbf{z} + \mathbf{F}_e(\mathbf{V}, \dot{\mathbf{V}}, \mathbf{\Omega}, \dot{\mathbf{\Omega}}, \mathbf{d}(t)) + R\Sigma_R\boldsymbol{\tau} + mg\mathbf{z} \tag{1.3}$$

where:

- $\mathbf{d}(t)$ are the external disturbances that do not depend on the vehicle's state.
- $\mathbf{F}_e$ are the external forces, which depend on the state.

- T is the **thrust**, which depends on the number and allocation of the rotors, while $-\mathbf{z}$ is the direction pointing upward (according to the NED frame as in Figure 1.2). R is the rotation matrix that describes the orientation of our vehicle with respect to the fixed NED frame SR_i.
- Hence, the product $-TR\mathbf{z}$ is the main upward thrust.
- $\boldsymbol{\tau}$ is the torque generated by the propellers.
- $R\Sigma_R\boldsymbol{\tau}$ are the parasitic forces generated by the rotors/actuators. The coupling matrix Σ_R maps the torques/rotations of the motors into small translational lateral forces (then rotated into SR_i via R).

We know that the derivative of the rotation matrix R depends on the skew-symmetric matrix of the angular velocity:

$$\dot{R} = R \cdot S(\boldsymbol{\Omega}) \tag{1.4}$$

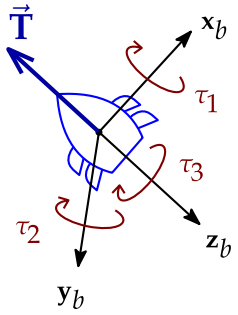
$S(\boldsymbol{\Omega})$ is important since it appears in the explicit expression of the term $J\dot{\boldsymbol{\Omega}}$ in Equation (1.2):

$$J\dot{\boldsymbol{\Omega}} = -S(\boldsymbol{\Omega})J\boldsymbol{\Omega} + \boldsymbol{\tau} + \boldsymbol{\tau}_e(\mathbf{V}, \dot{\mathbf{V}}, \boldsymbol{\Omega}, \dot{\boldsymbol{\Omega}}, \mathbf{d}(t)) + \boldsymbol{\tau}_g + \Sigma_T T\mathbf{z} \tag{1.5}$$

where:

- $\boldsymbol{\tau}_e$ are the torques induced by the external forces.
- $\Sigma_T T\mathbf{z}$ is a parasitic moment. Σ_T is a matrix that represents the geometric misalignment between the effective axis of the total thrust T and the center of mass of the vehicle. If the thrust vector does not pass exactly through the center of gravity, it generates a parasitic torque proportional to the thrust itself.
- $\boldsymbol{\tau}_g$ is the internal gyroscopic moment due to the inertia of the propellers rotating at high speed (particularly relevant in quadrotors when they change attitude rapidly).

We denote $\vec{\mathbf{T}} = -T\mathbf{z}_b$ the push in the opposite direction of $\mathbf{z}_b$ expressed in the vehicle frame SR_b.



A quadrotor is a UAV with four motors located at the extremities of a cross-shaped frame (Figure 1.4) and is controlled by imposing a desired angular speed on each rotor.

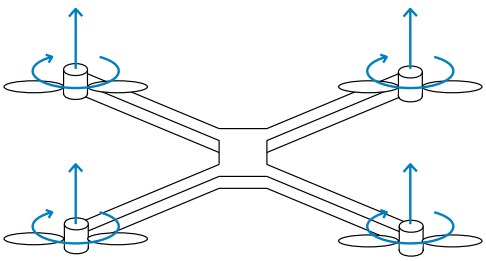


Figure 1.4: Schematic of a quadrotor.

Let's identify a disc on each rotor, which follows the profile of the propellers: we denote by f_i the scalar force on each propeller, which is perpendicular to the disc; its value is:

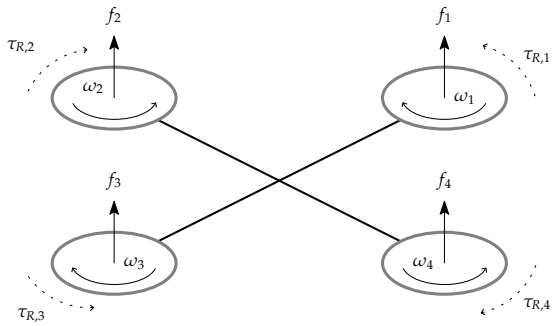
$$f_i = b\omega_i^2 \tag{1.6}$$

where ω_i is the angular speed of the i-th propeller, and b is a constant *thrust factor* that depends on the rotor geometry and profile, and on air density. The rotational speed of each blade is controlled by a low-level controller.

We denote $\tau_{R,i}$ the drag torque generated by the i-th rotor as it spins in the air:

$$\tau_{R,i} = d\omega_i^2 \tag{1.7}$$

where d is a *drag factor* that, like the thrust factor, depends on the rotor.



We denote φ, ϑ, ψ the roll-pitch-yaw angles of the vehicle with respect to the fixed reference frame SR_i, hence the already mentioned rotation matrix is:

$${}^iR_b = \begin{pmatrix} \cos\psi\cos\vartheta & \cos\psi\sin\vartheta\sin\varphi - \sin\psi\cos\varphi & \cos\psi\sin\vartheta\cos\varphi + \sin\psi\sin\varphi \\ \sin\psi\cos\vartheta & \sin\psi\sin\vartheta\sin\varphi + \cos\psi\cos\varphi & \sin\psi\sin\vartheta\cos\varphi - \sin\varphi\cos\psi \\ -\sin\vartheta & \cos\vartheta\sin\varphi & \cos\vartheta\cos\varphi \end{pmatrix}. \tag{1.8}$$

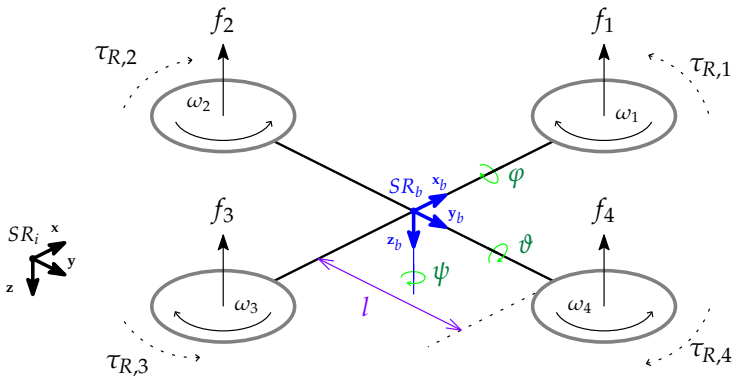


Figure 1.5: Configuration of the quadrotor.

The linear velocity $\mathbf{V}$ of our quadrotor is expressed in SR_i while the angular velocity $\mathbf{\Omega}$ is expressed in SR_b, we can make the reference system explicit using superscripts as follows:

$${}^i\mathbf{V} = \begin{pmatrix} v_x \\ v_y \\ v_z \end{pmatrix}, \quad {}^b\mathbf{\Omega} = \begin{pmatrix} p \\ q \\ r \end{pmatrix}. \tag{1.9}$$

Let $\omega_1, \omega_2, \omega_3, \omega_4$ be our control inputs (the velocity of the rotors), f_i and $\tau_{R,i}$ are derived from these quantities as in Equations (1.6) and (1.7). The total thrust will be:

$$T = f_1 + f_2 + f_3 + f_4. \tag{1.10}$$

We recall that on the quadrotor, a force $\vec{\mathbf{T}} = -T\mathbf{z}_b$ acts in the same direction as $\mathbf{z}_b$ but with the opposite sense. Intuitively, if the forces f_i are identical, we can expect the quadrotor to accelerate towards $-\mathbf{z}_b$ without rotating on itself. If the forces are not identical, they may cause a rotation of the body. The

torques generated on the axes $\mathbf{x}_b, \mathbf{y}_b, \mathbf{z}_b$ are the following:

$$\tau_\varphi = l(f_2 - f_4), \tag{1.11}$$
$$\tau_\vartheta = l(f_1 - f_3), \tag{1.12}$$
$$\tau_\psi = -\tau_{R,1} + \tau_{R,2} - \tau_{R,3} + \tau_{R,4}. \tag{1.13}$$

l represents the arm length (or lever arm) of the quadrotor, as shown in Figure 1.5.

For simplicity, we can assume that the inertia matrix shown in Equation (1.2) is diagonal:

$$J = \begin{pmatrix} I_x & 0 & 0 \\ 0 & I_y & 0 \\ 0 & 0 & I_z \end{pmatrix} \tag{1.14}$$

the angular velocity instead can be expressed as a function of the roll-pitch-yaw angles:

$$\boldsymbol{\Omega} = \begin{pmatrix} p \\ q \\ r \end{pmatrix} = \begin{pmatrix} 1 & 0 & -\sin\vartheta \\ 0 & \cos\varphi & \cos\vartheta\sin\varphi \\ 0 & -\sin\varphi & \cos\vartheta\cos\varphi \end{pmatrix} \begin{pmatrix} \dot{\varphi} \\ \dot{\vartheta} \\ \dot{\psi} \end{pmatrix}. \tag{1.15}$$

Let's see in detail what the applied forces and moments are. We write explicitly the sum of forces of the translational dynamics (that we recall is expressed in the fixed frame SR_i):

$$\sum_j \mathbf{F}_j = \underbrace{\begin{pmatrix} 0 \\ 0 \\ mg \end{pmatrix}}_{\text{weight}} + \underbrace{{}^iR_b \begin{pmatrix} 0 \\ 0 \\ -T \end{pmatrix}}_{\text{actuation}} + \mathbf{F}_A + \mathbf{F}_D \tag{1.16}$$

where $\mathbf{F}_A$ are the aerodynamics-related forces, which mainly include the aerodynamic resistance of the body and arms of the aircraft to advancement, as well as any parasitic lift effects generated by the shape of the drone or by the rotor discs in translation, while $\mathbf{F}_D$ are the disturbances (for example, wind).

The sum of moments is the following:

$$\sum_j \mathbf{M}_j = \underbrace{\begin{pmatrix} L_\varphi \\ L_\vartheta \\ L_\psi \end{pmatrix}}_{\text{gyroscopic effects}} + \underbrace{\begin{pmatrix} \tau_\varphi \\ \tau_\vartheta \\ \tau_\psi \end{pmatrix}}_{\text{actuation}} + \boldsymbol{\tau}_A + \boldsymbol{\tau}_D. \tag{1.17}$$

$\boldsymbol{\tau}_A$ is the resulting torque induced by the distributed aerodynamic forces when they do not have an application point coinciding with the center of gravity of the vehicle, combined with the rotational drag that the air exerts against the rotations of the rigid body in roll, pitch and yaw. $\boldsymbol{\tau}_D$ is the moment produced by the disturbances.

Definition 1.1.1 : State of the Quadrotor

Given the previous premises, we define the state of a quadrotor as a vector in $\mathbb{R}^{12}$ defined as follows:

$$\boldsymbol{\xi} = (x, \quad y, \quad z, \quad v_x, \quad v_y, \quad v_z, \quad \varphi, \quad \vartheta, \quad \psi, \quad p, \quad q, \quad r)^\top \tag{1.18}$$

Here x, y, z are the coordinates of the position of the center of mass of the quadrotor expressed in SR_i, v_x, v_y, v_z its velocity, φ, ϑ, ψ the roll-pitch-yaw angles between SR_i and SR_b, and p, q, r the angular velocity of the quadrotor expressed in SR_b.

The evolution in time of the quadrotor state is given by a dynamical system of the following form:

$$\dot{\boldsymbol{\xi}} = f(\boldsymbol{\xi}) + g(\boldsymbol{\xi})\mathbf{u} \tag{1.19}$$

where $\mathbf{u}$ are the inputs of the system defined as follows:

$$\mathbf{u} = \begin{pmatrix} T \\ \tau_\varphi \\ \tau_\vartheta \\ \tau_\psi \end{pmatrix} \tag{1.20}$$

these quantities depend on f_i, $\tau_{R,i}$, which in turn depend on the angular velocity of the rotors ω_i, which we assume we can control perfectly.

Definition 1.1.2 : Time Evolution of the Quadrotor

Let Ω_r be the the signed residual speed of the blades

$$\Omega_r = -\omega_1 + \omega_2 - \omega_3 + \omega_4$$

and I_r the inertia of the blades; the explicit expression of the system in Equation (1.19) is the following:

$$\dot{x} = v_x \tag{1.21}$$
$$\dot{y} = v_y \tag{1.22}$$
$$\dot{z} = v_z \tag{1.23}$$
$$\dot{v}_x = F_{A,x} - (\cos(\psi)\sin(\vartheta)\cos(\varphi) + \sin(\psi)\sin(\varphi))\frac{T}{m} \tag{1.24}$$
$$\dot{v}_y = F_{A,y} - (\sin(\psi)\sin(\vartheta)\cos(\varphi) - \sin(\varphi)\cos(\psi))\frac{T}{m} \tag{1.25}$$
$$\dot{v}_z = F_{A,z} + g - \cos(\vartheta)\cos(\varphi)\frac{T}{m} \tag{1.26}$$
$$\dot{\varphi} = p + \sin(\varphi)\tan(\vartheta)q + \cos(\varphi)\tan(\vartheta)r \tag{1.27}$$
$$\dot{\vartheta} = \cos(\varphi)q - \sin(\varphi)r \tag{1.28}$$
$$\dot{\psi} = \sin(\varphi)\sec(\vartheta)q + \cos(\varphi)\sec(\vartheta)r \tag{1.29}$$
$$\dot{p} = \tau_{A,x} + \frac{I_r}{I_x}q\Omega_r + \frac{I_y - I_z}{I_x}qr + \frac{\tau_\varphi}{I_x} \tag{1.30}$$
$$\dot{q} = \tau_{A,y} - \frac{I_r}{I_y}p\Omega_r + \frac{I_z - I_x}{I_y}pr + \frac{\tau_\vartheta}{I_y} \tag{1.31}$$
$$\dot{r} = \tau_{A,z} + \frac{I_x - I_y}{I_z}pq + \frac{\tau_\psi}{I_z}. \tag{1.32}$$

1.2 Quadrotor Control

The model in Definition 1.1.2 is too complex for control purposes. We neglect aerodynamics and gyroscopic effects, and we assume small variations of φ and ϑ such that

$$\begin{pmatrix}\dot{\varphi} & \dot{\vartheta} & \dot{\psi}\end{pmatrix}^\top \simeq \begin{pmatrix}p & q & r\end{pmatrix}^\top \tag{1.33}$$

we assume that the shape is symmetric, and we neglect the disturbances.

Definition 1.2.1 : Simplified Time Evolution of the Quadrotor

$$\ddot{x} = -(\cos(\psi)\sin(\vartheta)\cos(\varphi) + \sin(\psi)\sin(\varphi))\frac{T}{m} \tag{1.34}$$

$$\ddot{y} = -(\sin(\psi)\sin(\vartheta)\cos(\varphi) - \sin(\varphi)\cos(\psi))\frac{T}{m} \tag{1.35}$$

$$\ddot{z} = -\cos(\vartheta)\cos(\varphi)\frac{T}{m} + g \tag{1.36}$$

$$\ddot{\varphi} = \frac{\tau_\varphi}{I_x} \tag{1.37}$$

$$\ddot{\vartheta} = \frac{\tau_\vartheta}{I_y} \tag{1.38}$$

$$\ddot{\psi} = \frac{\tau_\psi}{I_z}. \tag{1.39}$$

Note how we changed the notation for the acceleration ($\ddot{x} = \dot{v}_x$, $\ddot{y} = \dot{v}_y$, $\ddot{z} = \dot{v}_z$).

1.2.1 Linear Control

Let's denote

$$\mathbf{p} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}, \quad \mathbf{\Theta} = \begin{pmatrix} \varphi \\ \vartheta \\ \psi \end{pmatrix}, \quad \mathbf{\Gamma} = \begin{pmatrix} \tau_\varphi \\ \tau_\vartheta \\ \tau_\psi \end{pmatrix} \tag{1.40}$$

we rewrite the state as

$$\boldsymbol{\xi} = \begin{pmatrix} \mathbf{p}^\top & \dot{\mathbf{p}}^\top & \mathbf{\Theta}^\top & \dot{\mathbf{\Theta}}^\top \end{pmatrix}^\top \tag{1.41}$$

with our input vector:

$$\mathbf{u} = \begin{pmatrix} T \\ \mathbf{\Gamma} \end{pmatrix}. \tag{1.42}$$

Let's separate the translational and rotational dynamics:

$$\ddot{\mathbf{p}} = g\mathbf{e}_3 - {}^iR_b(\mathbf{\Theta})\frac{T}{m}\mathbf{e}_3 \tag{1.43}$$

$$\ddot{\mathbf{\Theta}} = J^{-1}\mathbf{\Gamma}. \tag{1.44}$$

$\mathbf{e}_3 = (0,\ 0,\ 1)^\top$ is the unit vector pointing downward, previously denoted $\mathbf{z}$. Managing the nonlinear dynamics is hard for the real-time controller. We apply linearization around an equilibrium point, in particular in a hovering condition. Let

$$\mathbf{p}_e = \begin{pmatrix} x_e \\ y_e \\ z_e \end{pmatrix}, \quad \mathbf{\Theta}_e = \begin{pmatrix} \varphi_e \\ \vartheta_e \\ \psi_e \end{pmatrix} \tag{1.45}$$

be our equilibrium point. We assume that our drone is perfectly horizontal, i.e., $\varphi_e = \vartheta_e = 0$ with also $\psi_e = 0$, and there are no active torques: $\mathbf{\Gamma} = \mathbf{0}$. The thrust perfectly balances the gravity:

$$T_e = mg. \tag{1.46}$$

The full *hovering equilibrium* condition is expressed as follows:

$$\mathbf{\Theta}_e = \mathbf{0}, \quad \mathbf{\Gamma} = \mathbf{0}, \quad T_e = mg, \quad \dot{\mathbf{p}}_e = \mathbf{0}, \quad \dot{\mathbf{\Theta}}_e = \mathbf{0}. \tag{1.47}$$

We denote $\ddot{\mathbf{p}} = \mathbf{F}(\mathbf{\Theta}, T)$ the translational part in Equation (1.43). We approximate a small variation of $\ddot{\mathbf{p}}$ with its first order expansion around the equilibrium point:

$$\delta\ddot{\mathbf{p}} \simeq \frac{\partial \mathbf{F}}{\partial \mathbf{\Theta}}\bigg|_e \delta\mathbf{\Theta} + \frac{\partial \mathbf{F}}{\partial T}\bigg|_e \delta T \tag{1.48}$$

with δT and $\delta\boldsymbol{\Theta}$ being small variations. To compute the expansion, we need to compute the variation of the rotation matrix ${}^{i}R_b(\boldsymbol{\Theta})$, given by the small variation $\delta\boldsymbol{\Theta}$ of the angles around $\boldsymbol{\Theta}_e = \mathbf{0}$. We have

$$\delta\,{}^{i}R_b = \frac{\partial {}^{i}R_b}{\partial\varphi}\Big|_{\mathbf{0}}\delta\varphi + \frac{\partial {}^{i}R_b}{\partial\vartheta}\Big|_{\mathbf{0}}\delta\vartheta + \frac{\partial {}^{i}R_b}{\partial\psi}\Big|_{\mathbf{0}}\delta\psi. \tag{1.49}$$

We recall that ${}^{i}R_b(\boldsymbol{\Theta}) = R_z(\psi)R_y(\vartheta)R_x(\varphi)$, hence (without showing the whole calculation):

$$\delta^{i}R_b = \begin{pmatrix} 0 & -\delta\psi & \delta\vartheta \\ \delta\psi & 0 & -\delta\varphi \\ -\delta\vartheta & \delta\varphi & 0 \end{pmatrix} \tag{1.50}$$

this is exactly the skew-symmetric matrix $S(\delta\boldsymbol{\Theta})$. Now we can make explicit the formula in Equation (1.48):

$$\frac{\partial \mathbf{F}}{\partial\boldsymbol{\Theta}}\Big|_e \delta\boldsymbol{\Theta} = -gS(\delta\boldsymbol{\Theta})\mathbf{e}_3 \tag{1.51}$$

$$\frac{\partial \mathbf{F}}{\partial T}\Big|_e \delta T = -\frac{\delta T}{m}\mathbf{e}_3 \tag{1.52}$$

$$\implies \delta\ddot{\mathbf{p}} \simeq -g\begin{pmatrix} \delta\vartheta \\ -\delta\varphi \\ 0 \end{pmatrix} - \frac{\delta T}{m}\mathbf{e}_3. \tag{1.53}$$

Now we drop the notation with the δ (small variations), the quadrotor system linearized around an equilibrium point is governed by the following equations:

$$\ddot{x} = -g\vartheta, \qquad \ddot{\vartheta} = \frac{\tau_\vartheta}{I_y}, \tag{1.54}$$

$$\ddot{y} = g\varphi, \qquad \ddot{\varphi} = \frac{\tau_\varphi}{I_x}, \tag{1.55}$$

$$\ddot{z} = -\frac{(T - mg)}{m}, \qquad \ddot{\psi} = \frac{\tau_\psi}{I_z}. \tag{1.56}$$

This system is linear and decoupled:

- In order to control x, we can control ϑ through τ_ϑ, hence x is controllable with a chain of four integrators.
- In order to control y, we can control φ through τ_φ, hence y is controllable with a chain of four integrators.
- In order to control z we use T in a chain of two integrators.
- In order to control ψ we use τ_ψ in a chain of two integrators.

1.2.2 Basic Control and Planning Problems

The basic control objectives for a quadrotor are the following:

- attitude control,
- height control,
- position control,
- hovering-to-hovering motion,
- waypoint navigation,
- trajectory planning and tracking,
- obstacle avoidance.

With **attitude control**, we mean the process of determining the torques $\tau_\varphi, \tau_\vartheta, \tau_\psi$ needed to obtain a stable desired attitude $\varphi_d, \vartheta_d, \psi_d$. A simple solution is a proportional-derivative (PD) action on the error.

Definition 1.2.2 : PD Controller for the Attitude

$$\tau_\varphi = K_{\varphi p}(\varphi_d - \varphi) + K_{\varphi d}(\dot{\varphi}_d - \dot{\varphi}) \tag{1.57}$$
$$\tau_\vartheta = K_{\vartheta p}(\vartheta_d - \vartheta) + K_{\vartheta d}(\dot{\vartheta}_d - \dot{\vartheta}) \tag{1.58}$$
$$\tau_\psi = K_{\psi p}(\psi_d - \psi) + K_{\psi d}(\dot{\psi}_d - \dot{\psi}). \tag{1.59}$$

For the **height control** we consider the dynamical model, in particular Equation (1.36). We choose the value of T that linearizes the system.

Definition 1.2.3 : Height Control with FBL

To bring the quadrotor to a desired height z_d and keep it there, we apply feedback linearization (FBL) plus a PD controller.

$$T = \frac{m}{\cos\vartheta\cos\varphi}[g - \ddot{z}_d - K_{zp}(z_d - z) - K_{zd}(\dot{z}_d - \dot{z})]. \tag{1.60}$$

Observation 1.2.1

A quadrotor is an underactuated system: it has 6 degrees of freedom in space (x, y, z and three attitude angles), but only 4 independent control inputs (the total thrust T and the three torque moments $\tau_\varphi, \tau_\vartheta, \tau_\psi$). There are no side motors to push the drone directly along the x axis or the y axis. To translate horizontally, the drone must first tilt (rotate) so that a component of the thrust T points in the desired direction.

Given the underactuated nature of the quadrotor, while we are controlling the altitude z we cannot directly control x and y, hence we adopt a *hierarchical* control scheme.

1. First, we assume that we have direct control on x and y, with two **virtual control inputs** u_x and u_y:

$$\ddot{x} = u_x \tag{1.61}$$
$$\ddot{y} = u_y. \tag{1.62}$$

2. These virtual control inputs are defined as a PD action on the error for stabilization:

$$u_x = \ddot{x}_d + K_{xp}(x_d - x) + K_{xd}(\dot{x}_d - \dot{x}) \tag{1.63}$$
$$u_y = \ddot{y}_d + K_{yp}(y_d - y) + K_{yd}(\dot{y}_d - \dot{y}). \tag{1.64}$$

3. From the dynamics in Equations (1.34) and (1.35), we denote T_x and T_y the normalized components of the thrust along the inertial axes x and y:

$$\ddot{x} = \underbrace{(-\cos\varphi\sin\vartheta\cos\psi - \sin\varphi\sin\psi)}_{T_x}\frac{T}{m} = T_x\frac{T}{m} \tag{1.65}$$
$$\ddot{y} = \underbrace{(-\cos\varphi\sin\vartheta\sin\psi + \sin\varphi\cos\psi)}_{T_y}\frac{T}{m} = T_y\frac{T}{m}. \tag{1.66}$$

4. Let's equate the real expressions to the virtual accelerations:

$$T_x \frac{T}{m} = u_x \implies T_x = \frac{m}{T} u_x = \frac{m}{T} \left[K_{xp}(x_d - x) + K_{xd}(\dot{x}_d - \dot{x}) + \ddot{x}_d \right] \tag{1.67}$$

$$T_y \frac{T}{m} = u_y \implies T_y = \frac{m}{T} u_y = \frac{m}{T} \left[K_{yp}(y_d - y) + K_{yd}(\dot{y}_d - \dot{y}) + \ddot{y}_d \right] \tag{1.68}$$

   in this way we get the desired value for T_x and T_y in order to achieve u_x and u_y.

5. At this point we have to solve (1.65) and (1.66) for the roll and the pitch to get the desired values φ_d and ϑ_d:

$$\varphi_d = \arcsin \left(T_y \cos\psi - T_x \sin\psi \right) \tag{1.69}$$

$$\vartheta_d = -\arcsin \left(\frac{T_x \cos\psi + T_y \sin\psi}{\cos\varphi_d} \right). \tag{1.70}$$

6. Finally, the inner loop will generate the desired values for $(\tau_\varphi, \tau_\vartheta, \tau_\psi)$ to achieve φ_d and ϑ_d.

To recap the whole control scheme:

- The quadrotor has no laterally oriented propellers, and generates a single main thrust T perpendicular to its frame. Consequently, there is no independent direct force on x and y.
- T is set as a priority to support the weight of the quadrotor and govern the z altitude.
- If we keep the drone perfectly horizontal, the thrust only points upwards. If, however, we tilt it in roll (φ) and pitch (ϑ), the thrust vector is decomposed: one part continues to compensate for gravity along z, while the other is projected onto the horizontal plane.
- The position loop calculates how much horizontal acceleration $(\ddot{x}, \ddot{y})$ is needed to reach the target and, working backwards, determines exactly which roll and pitch angles (φ_d, ϑ_d) project the necessary amount of thrust T in that precise direction.

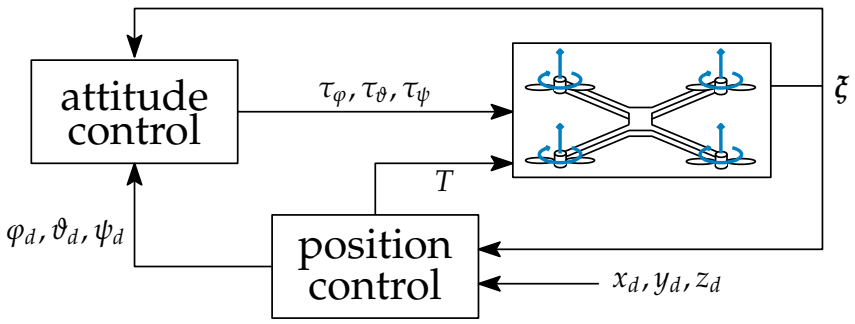


Figure 1.6: Structure of the control system.

CHAPTER

2

CONTROL METHODS

2.1 Backstepping

For a dynamical system:

$$\begin{cases} \dot{\mathbf{x}} = f(\mathbf{x}) \\ f(\mathbf{0}) = \mathbf{0} \end{cases} \tag{2.1}$$

we recall that a Lyapunov function is a scalar function that depends on $\mathbf{x}$, is continuously differentiable and positive definite:

$$V(\mathbf{x}) > 0, \quad \forall \mathbf{x} \neq \mathbf{0} \tag{2.2}$$

$$V(\mathbf{0}) = 0. \tag{2.3}$$

If $\dot{V} = \frac{\partial V}{\partial \mathbf{x}} f(\mathbf{x})$ is negative definite, then the equilibrium $\mathbf{x} = \mathbf{0}$ is asymptotically stable. We also know that:

- If V is radially unbounded, the system is globally asymptotically stable.
- If $\exists c_1 > 0, c_2 > 0, c_3 > 0$ with $c_1 \leq c_2$ such that

  $$c_1 \|\mathbf{x}\|^2 \leq V \leq c_2 \|\mathbf{x}\|^2, \tag{2.4}$$

  $$\dot{V} \leq -c_3 \|\mathbf{x}\|^2 \tag{2.5}$$

  the system is globally exponentially stable.
- If $\dot{V}$ is negative semidefinite, the asymptotic stability can be proved using LaSalle's invariance principle.

In general, V is hard to find, so the backstepping method is used to construct both V and the control law; it renders an equilibrium point or a trajectory of a nonlinear system globally asymptotically stable (GAS).

The assumption is that our system is in strict-feedback (triangular) form; the quadrotor model satisfies this assumption. We will construct a simple two-dimensional example of how backstepping works. We have the following nonlinear system:

$$\dot{x}_1 = f_1(x_1) + g_1(x_1) x_2 \tag{2.6}$$

$$\dot{x}_2 = f_2(x_1, x_2) + g_2(x_1, x_2) u \tag{2.7}$$

with $x_1 = 0$ being an equilibrium for the first subsystem. Notice the triangular structure: the first subsystem that describes the dynamics of x_1 depends nonlinearly only on x_1, and linearly on x_2. The second subsystem depends linearly on u (control input) and nonlinearly on (x_1, x_2). We also assume that

$$\begin{matrix} g_1(x_1) \neq 0 \\ g_2(x_1, x_2) \neq 0 \end{matrix} \qquad \forall (x_1, x_2). \tag{2.8}$$

- **Step 1**: We stabilize x_1 by assuming that x_2 is a control input (can be chosen arbitrarily):

$$x_2 = \alpha(x_1) = \frac{1}{g_1(x_1)}[-f_1(x_1) - k_1 x_1] \tag{2.9}$$

with the Lyapunov function $V_1 = \frac{1}{2}x_1^2$. This leads to a simple differential equation:

$$\dot{x}_1 = -k_1 x_1 \tag{2.10}$$

and

$$\dot{V}_1 = -k_1 x_1^2 \tag{2.11}$$

hence the controller will globally stabilize x_1 to 0.

- **Step 2**: In reality, x_2 has its own dynamics and is not necessarily equal to our *virtual* control input $\alpha(x_1)$, hence we define an error:

$$z = x_2 - \alpha(x_1) \tag{2.12}$$

the dynamics of this error is

$$\dot{z} = \dot{x}_2 - \dot{\alpha} \tag{2.13}$$

we substitute $\dot{x}_2$ with its dynamics:

$$\dot{z} = f_2 + g_2 u - \frac{\partial \alpha}{\partial x_1}\dot{x}_1 \tag{2.14}$$

notice how we dropped the dependencies (for the sake of notational simplicity). Now we substitute $\dot{x}_1$ with its dynamics:

$$\dot{z} = f_2 + g_2 u - \frac{\partial \alpha}{\partial x_1}(f_1 + g_1 x_2). \tag{2.15}$$

For the system in Equation (2.15) we consider the following Lyapunov function:

$$V = \frac{1}{2}x_1^2 + \frac{1}{2}z^2$$

that is zero only at $x_1 = 0, z = 0$. We want to stabilize z to 0 (nullify the error on x_2). The derivative is:

$$\dot{V} = -k_1 x_1^2 + x_1 g_1 z + z(f_2 + g_2 u - \dot{\alpha}). \tag{2.16}$$

To reach the desired behavior, we choose u (our real control input) in order to make $\dot{V}$ negative definite:

$$u = \frac{1}{g_2}[-f_2 + \dot{\alpha} - k_2 z - x_1 g_1] \implies \tag{2.17}$$

$$\dot{V} = -k_1 x_1^2 - k_2 z^2 \leq -2\min(k_1, k_2)V \tag{2.18}$$

with k_1, k_2 being two positive real numbers. The term $-k_2 z$ drives x_2 to $\alpha(x_1)$. This control law achieves exponential stability for the system in x_1 and z, and global asymptotic stability in x_1 and x_2. The achieved closed loop system in the error coordinates is

$$\begin{pmatrix} \dot{x}_1 \\ \dot{z} \end{pmatrix} = \begin{pmatrix} -k_1 & g_1 \\ -g_1 & -k_2 \end{pmatrix} \begin{pmatrix} x_1 \\ z \end{pmatrix}. \tag{2.19}$$

The backstepping method can be recursively applied to a general triangular n-th order system:

$$\dot{x}_i = f_i(x_1, \ldots, x_i) + g_i(x_1, \ldots, x_i)x_{i+1}, \qquad i = 1, \ldots, n-1 \tag{2.20}$$

$$\dot{x}_n = f_n(\mathbf{x}) + g_n(\mathbf{x})u. \tag{2.21}$$

For each step we define the error z_i, with the addition of a term to the Lyapunov function:

$$\dot{z}_i = -k_i z_i - g_{i-1} z_{i-1} + g_i z_{i+1} \tag{2.22}$$
$$\dot{V} = -\sum_i k_i z_i^2. \tag{2.23}$$

The backstepping method is a design procedure, not a defined control law, and it does not need to cancel the nonlinearities of a system to achieve stability. The problem is that the complexity of this procedure grows exponentially since each step needs the analytical derivative of the previous virtual input.

2.1.1 Backstepping for Height Control

Let's see an application of backstepping. The height dynamics of the quadrotor is the following:

$$\dot{z} = v_z \tag{2.24}$$
$$\dot{v}_z = g - \cos\varphi \cos\vartheta \frac{T}{m} \tag{2.25}$$

we choose T in order to compensate for gravity and linearize the system:

$$T = \frac{m}{\cos\varphi\cos\vartheta}(g - \tilde{T}) \implies \dot{v}_z = \tilde{T} \tag{2.26}$$

with $\tilde{T}$ to be chosen later. Given the desired height $z_d(t)$ we define the error:

$$e_z = z_d - z \implies \dot{e}_z = \dot{z}_d - v_z. \tag{2.27}$$

If the velocity v_z were the control input, we would choose:

$$v_z = K_{zp} e_z + \dot{z}_d \implies \dot{e}_z = -K_{zp} e_z \tag{2.28}$$

but v_z has its own dynamics, hence we consider the formula in Equation (2.28) as a virtual control input denoted v_z^d:

$$v_z^d = K_{zp} e_z + \dot{z}_d \tag{2.29}$$

so we can define the error between the virtual input and the real velocity:

$$e_{v_z} = v_z^d - v_z \tag{2.30}$$

furthermore:

$$\dot{e}_z = -K_{zp} e_z + e_{v_z} \tag{2.31}$$
$$\dot{e}_{v_z} = K_{zp}\dot{e}_z + \ddot{z}_d - \tilde{T}. \tag{2.32}$$

The Lyapunov function becomes:

$$V = \frac{1}{2}(e_z^2 + e_{v_z}^2) \tag{2.33}$$
$$\dot{V} = -K_{zp} e_z^2 + e_{v_z}(e_z + K_{zp}\dot{e}_z + \ddot{z}_d - \tilde{T}) \tag{2.34}$$

now we choose $\tilde{T}$ to make $\dot{V}$ negative definite

$$\tilde{T} = \ddot{z}_d + e_z + K_{zp}\dot{e}_z + K_{zd} e_{v_z} \implies \tag{2.35}$$
$$\dot{V} = -K_{zp} e_z^2 - K_{zd} e_{v_z}^2 \tag{2.36}$$

by putting everything together, the final control law is:

$$T = \frac{m}{\cos\varphi\cos\vartheta}\left(g - \ddot{z}_d - (1 + K_{zp}K_{zd})e_z - (K_{zp} + K_{zd})\dot{e}_z\right). \tag{2.37}$$

2.1.2 PD Compared With Backstepping

What are the differences between the PD seen in Section 1.2.2 and the backstepping? They share a similar structure, but with a different choice of coordinates. The control laws are identical if we set the gain of the PD as follows:

$$K_p = 1 + K_{zp}K_{zd} \tag{2.38}$$

$$K_d = K_{zp} + K_{zd}. \tag{2.39}$$

The PD controller aims to nullify the position and velocity errors $(e_z, \dot{e}_z)$, while the backstepping law considers $e_{v_z} = \dot{e}_z + K_{zp}e_z$ instead of $\dot{e}_z$, hence the desired velocity already contains a term related to the position correction.

The gains have different roles:

- K_{zp} controls the decay rate of the position error, when the velocity error is zero.
- K_{zd} is the decay rate of the velocity error (the inner loop of the backstepping).
- In the PD controller, the natural frequency and the damping ratio of the error dynamics are given by

$$\omega_n = \sqrt{K_p} \tag{2.40}$$

$$\zeta = \frac{K_d}{2\sqrt{K_p}}. \tag{2.41}$$

With the same choices for the gains $K_p = K_{zp}, K_d = K_{zd}$ the closed-loop poles are (by setting $K_p = K_d = 2$):

- **backstepping**: $-2 \pm i$
- **PD**: $-1 \pm i$ (twice as slow).

A PD controller can be obtained from the backstepping procedure. The backstepping provides a hierarchical tuning procedure, with an explicit Lyapunov function.

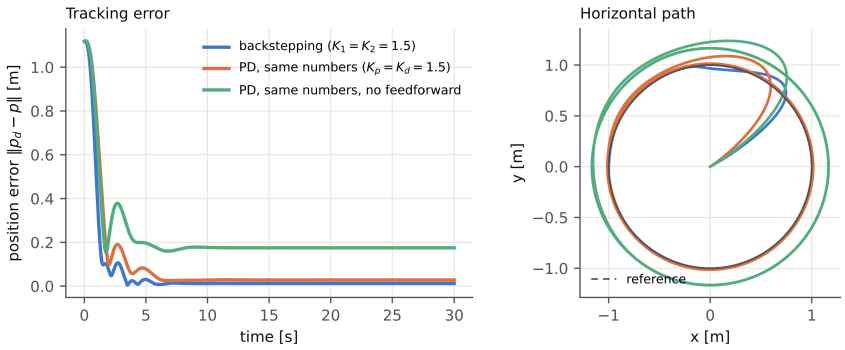


Figure 2.1: Simulated comparison between backstepping and PD.

2.1.3 Integral Backstepping

Now we consider the roll dynamics:

$$\ddot{\varphi} = \frac{\tau_\varphi}{I_x} + h_\varphi \tag{2.42}$$

h_φ represents the nonlinear terms, for example the gyroscopic effects. We define the integral action of the error:

$$e_1 = \varphi_d - \varphi, \tag{2.43}$$

$$\chi = \int_0^t e_1 ds \tag{2.44}$$

then we define the desired (or virtual) roll velocity as:

$$\omega_d = c_1 e_1 + \dot{\varphi}_d + \lambda\chi \tag{2.45}$$

with c_1 and λ being control gains. We define a second error on the desired velocity:

$$e_2 = \omega_d - \dot{\varphi} \tag{2.46}$$

by deriving with respect to time the error equations we get:

$$\dot{e}_1 = -c_1 e_1 - \lambda\chi + e_2 \tag{2.47}$$
$$\dot{e}_2 = c_1 \dot{e}_1 + \ddot{\varphi}_d + \lambda e_1 - \ddot{\varphi} \tag{2.48}$$

to guarantee the asymptotic stability of the system we want to impose the following behavior for the error:

$$\dot{e}_2 = -c_2 e_2 - e_1 \tag{2.49}$$

hence we choose the following control law:

$$\tau_\varphi = I_x \left[(1 - c_1^2 + \lambda) e_1 + (c_1 + c_2) e_2 - c_1 \lambda \chi + \ddot{\varphi}_d - h_\varphi \right] \tag{2.50}$$

the same construction can be done for the altitude and the position. The stability of the system with this control law can be proved by defining the following Lyapunov candidate:

$$V = \frac{1}{2}\lambda\chi^2 + \frac{1}{2}e_1^2 + \frac{1}{2}e_2^2 \tag{2.51}$$
$$\dot{V} = -c_1 e_1^2 - c_2 e_2^2 \tag{2.52}$$

on the equilibrium $e_1 = e_2 = 0$ the system becomes:

$$\dot{e}_1 = -\lambda\chi \tag{2.53}$$

that is zero when $\chi = 0$ (GAS proved by LaSalle).

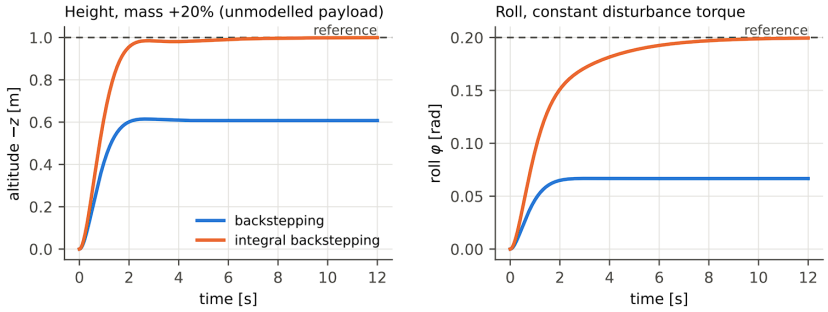


Figure 2.2: Performance comparison between backstepping and integral backstepping controllers, demonstrating that the integral action successfully eliminates steady-state errors under unmodeled payload (left) and constant disturbance torque (right).

2.2 Dynamic Feedback Linearization

One of the most important and effective ways to control a nonlinear dynamical system is to make it linear through feedback.

Definition 2.2.1 : Feedback Linearization

Feedback linearization is the exact cancellation of the nonlinearities by feedback and change of coordinates.

Recap of the model: For control purposes, we are using the (already seen) simplified model that assumes small changes for φ and ϑ

$$(\dot{\varphi}, \dot{\vartheta}, \dot{\psi}) \simeq (p, q, r)$$

and negligible aerodynamics/gyroscopic effects:

$$\ddot{x} = -(\cos\psi \sin\vartheta \cos\varphi + \sin\psi \sin\varphi)\frac{T}{m} \tag{2.54}$$

$$\ddot{y} = -(\sin\psi \sin\vartheta \cos\varphi - \sin\varphi \cos\psi)\frac{T}{m} \tag{2.55}$$

$$\ddot{z} = -\cos\vartheta \cos\varphi \frac{T}{m} + g \tag{2.56}$$

$$\ddot{\varphi} = \frac{\tau_\varphi}{I_x} \tag{2.57}$$

$$\ddot{\vartheta} = \frac{\tau_\vartheta}{I_y} \tag{2.58}$$

$$\ddot{\psi} = \frac{\tau_\psi}{I_z}. \tag{2.59}$$

We have used the symbol $\boldsymbol{\xi}$ for the state and $\mathbf{u}$ for the inputs. The compact form is

$$\begin{cases} \ddot{\mathbf{p}} = g\mathbf{e}_3 - \frac{T}{m}\mathbf{b}(\boldsymbol{\Theta}) \\ \ddot{\boldsymbol{\Theta}} = J^{-1}\boldsymbol{\Gamma}. \end{cases} \tag{2.60}$$

where we defined:

$$\mathbf{b}(\boldsymbol{\Theta}) = {}^iR_b(\boldsymbol{\Theta})\mathbf{e}_3 \tag{2.61}$$

$$\mathbf{p} = \begin{pmatrix} x & y & z \end{pmatrix}^\top \tag{2.62}$$

$$\boldsymbol{\Theta} = \begin{pmatrix} \varphi & \vartheta & \psi \end{pmatrix}^\top \tag{2.63}$$

$$\boldsymbol{\Gamma} = \begin{pmatrix} \tau_\varphi & \tau_\vartheta & \tau_\psi \end{pmatrix}^\top \tag{2.64}$$

$$J = \text{diag}(I_x, I_y, I_z). \tag{2.65}$$

2.2.1 Static Linearization

Before discussing the linearization of the quadrotor model, we will give some definitions for a general nonlinear system, but keep in mind the model in Equation (2.60), as we will need it later.

Consider the following nonlinear system:

$$\dot{\boldsymbol{\xi}} = f(\boldsymbol{\xi}) + g(\boldsymbol{\xi})\mathbf{u} \qquad \begin{matrix} \boldsymbol{\xi} \in \mathbb{R}^n \\ \mathbf{u} \in \mathbb{R}^m. \end{matrix} \tag{2.66}$$

If possible, we *want to find* a **static** state feedback and a change of coordinates:

$$\mathbf{u} = \alpha(\boldsymbol{\xi}) + \beta(\boldsymbol{\xi})\mathbf{v} \tag{2.67}$$

$$\mathbf{z} = \phi(\boldsymbol{\xi}) \tag{2.68}$$

with β a non-singular function and ϕ a diffeomorphism, such that the closed-loop system becomes linear and controllable:

$$\dot{\boldsymbol{\xi}} = f(\boldsymbol{\xi}) + g(\boldsymbol{\xi})\mathbf{u} \implies \tag{2.69}$$

$$\dot{\boldsymbol{\xi}} = f(\boldsymbol{\xi}) + g(\boldsymbol{\xi})(\alpha(\boldsymbol{\xi}) + \beta(\boldsymbol{\xi})\mathbf{v}) \tag{2.70}$$

since $\boldsymbol{\xi} = \phi^{-1}(\mathbf{z})$:

$$\frac{d}{dt}\phi^{-1}(\mathbf{z}) = f(\phi^{-1}(\mathbf{z})) + g(\phi^{-1}(\mathbf{z}))(\alpha(\phi^{-1}(\mathbf{z})) + \beta(\phi^{-1}(\mathbf{z}))\mathbf{v}) \implies \tag{2.71}$$

$$\dot{\mathbf{z}} = A\mathbf{z} + B\mathbf{v} \tag{2.72}$$

with A and B being constant matrices. This linearization is exact and holds whenever β and ϕ are well defined; the control design is completed by choosing $\mathbf{v}$ through the rules of linear control theory, and since ϕ is a diffeomorphism, by controlling $\mathbf{z}$ we can indirectly control $\boldsymbol{\xi}$.

2.2.2 Input-Output Linearization

With this method, we focus on a subset of the state vector that we want to control, by allowing (in general) a residual internal dynamics on the remaining variables. We choose m variables (same number of inputs) with a (sufficiently smooth) function h:

$$\boldsymbol{\eta} = h(\boldsymbol{\xi}) \in \mathbb{R}^m \tag{2.73}$$

We know that the inputs $\mathbf{u}$ are present in $\dot{\boldsymbol{\xi}}$, hence by differentiating $h(\boldsymbol{\xi})$, at some point one or more inputs of $\mathbf{u}$ will appear. $\boldsymbol{\eta}$ is a vector of functions η_i, $1 \leq i \leq m$, we have to differentiate each of these functions until *at least* one of the inputs of $\mathbf{u}$ appears.

Algorithm 1 Input-Output Linearization

```
for i = 1, ..., m do
    found ← false
    for r_i = 1, ..., +∞ do
        for j = 1, ..., m do
            if u_j is present in η_i^(r_i) then
                save η_i^(r_i)
                found ← true
                break
            end if
        end for
        if found then
            break
        end if
    end for
end for
```

The explicit expression of $\eta_i^{(r_i)}$ is

$$\eta_i^{(r_i)} = \underbrace{L_f^{r_i} h_i(\boldsymbol{\xi})}_{\text{Lie derivative}} + \sum_{j=1}^{m} L_{g_j} L_f^{r_i - 1} h_i(\boldsymbol{\xi}) u_j. \tag{2.74}$$

where r_i is the relative degree of differentiation for the i-th component of $\boldsymbol{\eta}$. Clearly, h should be in the class C^r with $r = \max_i r_i$. The Lie derivatives are defined as follows:

$$L_f^0 h_i(\boldsymbol{\xi}) = h_i(\boldsymbol{\xi}) \tag{2.75}$$

$$L_f^1 h_i(\boldsymbol{\xi}) = L_f h_i(\boldsymbol{\xi}) = \frac{\partial h_i(\boldsymbol{\xi})}{\partial \boldsymbol{\xi}} f(\boldsymbol{\xi}) \tag{2.76}$$

$$L_f^k h_i(\boldsymbol{\xi}) = \frac{\partial (L_f^{k-1} h_i)}{\partial \boldsymbol{\xi}} f(\boldsymbol{\xi}). \tag{2.77}$$

$L_{g_j} L_f^{r_i - 1} h_i(\boldsymbol{\xi})$ represents the Lie derivative of the intermediate scalar function $L_f^{r_i - 1} h_i(\boldsymbol{\xi})$ along the input vector field $g_j(\boldsymbol{\xi})$:

$$L_{g_j} L_f^{r_i - 1} h_i(\boldsymbol{\xi}) = \frac{\partial (L_f^{r_i - 1} h_i(\boldsymbol{\xi}))}{\partial \boldsymbol{\xi}} g_j(\boldsymbol{\xi}). \tag{2.78}$$

About the Notation 2.2.1

When the superscript of a symbol is enclosed in round brackets, it denotes a derivative; if there are no brackets, it is an exponent or an index:

a^k is a raised to the power k (2.79)

$a^{(k)}$ is the k-th derivative of the function a. (2.80)

For the particular case of the Lie derivatives, the superscript k in L_f^k does not represent an exponent but an order of differentiation, even though it is not enclosed in brackets.

We can stack the m equations defined in (2.74) and write the system in compact form:

$$\begin{pmatrix} \eta_1^{(r_1)} \\ \vdots \\ \eta_m^{(r_m)} \end{pmatrix} = I(\boldsymbol{\xi}) + D(\boldsymbol{\xi})\mathbf{u} \tag{2.81}$$

$D(\boldsymbol{\xi})$ is called the **decoupling matrix**. If it is non-singular, then, for $\mathbf{v} \in \mathbb{R}^m$, choosing the following control law:

$$\mathbf{u} = D^{-1}(\boldsymbol{\xi})(-I(\boldsymbol{\xi}) + \mathbf{v}) \tag{2.82}$$

we achieve the following dynamic behavior:

$$\eta_i^{(r_i)} = v_i, \quad i = 1, \dots, m \tag{2.83}$$

we obtain m **decoupled chains of integrators** from v_i to η_i.

- if $\sum_i r_i = n$, we have **full state linearization**, and $\mathbf{z} = (\eta_i, \eta_i^{(1)}, \dots, \eta_i^{(r_i-1)})$, for $i = 1, \dots, m$, are coordinates.
- if $\sum_i r_i < n$, there will be an $(n - \sum_i r_i)$-dimensional internal dynamics, whose stability must be checked separately.

2.2.3 Linearization of the Quadrotor Model

From the quadrotor model in Equation (2.60) we consider as natural outputs the position and the yaw angle:

$$\boldsymbol{\eta} = \begin{pmatrix} x & y & z & \psi \end{pmatrix}^\top \tag{2.84}$$

the model relative only to these variables is:

$$\ddot{\boldsymbol{\eta}} = \begin{pmatrix} \ddot{\mathbf{p}} \\ \ddot{\psi} \end{pmatrix} = \begin{pmatrix} g\mathbf{e}_3 \\ 0 \end{pmatrix} + \begin{pmatrix} -\frac{1}{m}\mathbf{b}(\boldsymbol{\Theta}) & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ 0 & 0 & 0 & \frac{1}{I_z} \end{pmatrix} \begin{pmatrix} T \\ \tau_\varphi \\ \tau_\vartheta \\ \tau_\psi \end{pmatrix}. \tag{2.85}$$

The quadrotor has six degrees of freedom and four control inputs; its underactuated nature makes it impossible to linearize the full state with a simple static feedback linearization $\mathbf{u} = \alpha(\boldsymbol{\xi}) + \beta(\boldsymbol{\xi})\mathbf{v}$. If we tried to linearize only the output statically, we would be left with non-linearized internal dynamics.

To overcome this problem, we must add a **dynamic compensator**, by introducing a set of ν additional states to the controller:

$$\boldsymbol{\zeta} \in \mathbb{R}^\nu \text{ (additional states)} \tag{2.86}$$

we have to specify a dynamics for these states:

$$\dot{\boldsymbol{\zeta}} = a(\boldsymbol{\xi}, \boldsymbol{\zeta}) + b(\boldsymbol{\xi}, \boldsymbol{\zeta})\mathbf{u} \tag{2.87}$$

defined by the functions a, b such that, together with the controller:

$$\mathbf{u} = c(\boldsymbol{\xi}, \boldsymbol{\zeta}) + d(\boldsymbol{\xi}, \boldsymbol{\zeta})\mathbf{v} \tag{2.88}$$

for some functions c, d the closed-loop system, of dimension $n + \nu$, is equivalent under a state transformation $\mathbf{z} = \phi(\boldsymbol{\xi}, \boldsymbol{\zeta})$ to a *linear controllable system*. Physically, in the quadrotor, the auxiliary state $\boldsymbol{\zeta}$ typically consists of integrators added on the thrust input T. By integrating the thrust twice, the true thrust becomes a state coordinate of the extended system, delaying the immediate appearance of the input into the derivatives and allowing the rotational dynamics to align with the translational ones without singularities.

Let's see the **algorithm of dynamic extension**: the construction procedure used to find the dynamic compensator and completely linearize the system.

1. We choose an m-dimensional output $\boldsymbol{\eta} = h(\boldsymbol{\xi})$.
2. We differentiate the outputs with respect to time until the inputs appear such that the decoupling matrix is non-singular (invertible).
3. When differentiating the components of $\boldsymbol{\eta}$, it often happens that some inputs appear at a lower differentiation order than others, making the decoupling matrix singular. We add integrators on those input channels: the inputs become states of the compensator, their derivatives become the new inputs.
4. If, at the end of the algorithm, the sum of the differentiation orders reached for each output equals the size of the extended state space:
$$\sum_{i=1}^{m} r_i = n + \nu$$
(where n is the size of the original physical state and ν is the total number of integrators added), *full input-state-output linearization* is obtained.
5. Applying the inversion law on the extended system, the behavior between the new inputs v_i and the physical outputs η_i is exactly equivalent to m independent and decoupled chains of pure integrators:
$$\eta_i^{(r_i)} = v_i, \quad i = 1, \dots, m. \tag{2.89}$$

The linearizing quadrotor output that we choose is the position together with the yaw angle:

$$\boldsymbol{\eta} = \begin{pmatrix} x & y & z & \psi \end{pmatrix}^\top \tag{2.90}$$

The model is the one in Equation (2.85). The inputs appear in a non-singular way only after differentiating η_1, η_2, η_3 four times and η_4 twice. The explicit expression of the vector $\mathbf{b}$ in the model is

$$\mathbf{b}(\boldsymbol{\Theta}) = {}^iR_b(\boldsymbol{\Theta})\mathbf{e}_3 = \begin{pmatrix} \cos\psi \sin\vartheta \cos\varphi + \sin\psi \sin\varphi \\ \sin\psi \sin\vartheta \cos\varphi - \cos\psi \sin\varphi \\ \cos\vartheta \cos\varphi \end{pmatrix} \tag{2.91}$$

we will need its Jacobian, denoted B:

$$B = \frac{\partial \mathbf{b}}{\partial \boldsymbol{\Theta}} = \begin{pmatrix} \sin\psi \cos\varphi - \cos\psi \sin\vartheta \sin\varphi & \cos\psi \cos\vartheta \cos\varphi & \cos\psi \sin\varphi - \sin\psi \sin\vartheta \cos\varphi \\ -\cos\psi \cos\varphi - \sin\psi \sin\vartheta \sin\varphi & \sin\psi \cos\vartheta \cos\varphi & \sin\psi \sin\varphi + \cos\psi \sin\vartheta \cos\varphi \\ -\cos\vartheta \sin\varphi & -\sin\vartheta \cos\varphi & 0 \end{pmatrix}. \tag{2.92}$$

By differentiating η_1, η_2, η_3 twice we get the translational part of the model:

$$\begin{pmatrix} \ddot{\eta}_1 & \ddot{\eta}_2 & \ddot{\eta}_3 \end{pmatrix}^\top = \ddot{\mathbf{p}} = g\mathbf{e}_3 - \frac{T}{m}\mathbf{b}(\boldsymbol{\Theta}) \tag{2.93}$$

we keep differentiating until the inputs $T, \boldsymbol{\Gamma}$ appear:

$$\mathbf{p}^{(3)} = -\frac{1}{m}(\dot{T}\mathbf{b} + TB\dot{\boldsymbol{\Theta}}) \tag{2.94}$$

one last differentiation:

$$\mathbf{p}^{(4)} = -\frac{1}{m}(\ddot{T}\mathbf{b} + 2\dot{T}B\dot{\boldsymbol{\Theta}} + T\dot{B}\dot{\boldsymbol{\Theta}} + TB\ddot{\boldsymbol{\Theta}}) \tag{2.95}$$

$\ddot{\boldsymbol{\Theta}}$ is directly mapped to $\boldsymbol{\Gamma}$ through the inertia matrix:

$$\mathbf{p}^{(4)} = -\frac{1}{m}(\ddot{T}\mathbf{b} + 2\dot{T}B\dot{\boldsymbol{\Theta}} + T\dot{B}\dot{\boldsymbol{\Theta}} + TBJ^{-1}\boldsymbol{\Gamma}) \tag{2.96}$$

finally, the inputs $\boldsymbol{\Gamma}$ appear in the Cartesian snap $\mathbf{p}^{(4)}$. Since $\eta_4 = \psi$, we need just two differentiations in order to get a non-singular dependence on τ_ψ:

$$\ddot{\psi} = \frac{\tau_\psi}{I_z}. \tag{2.97}$$

In order to control $\mathbf{p}^{(4)}$ we have to choose $\ddot{T}$, hence this will be our input, while T and $\dot{T}$ become states:

$$\zeta_1 = T, \quad \zeta_2 = \dot{T} \tag{2.98}$$

the model becomes:

$$\begin{pmatrix} \mathbf{p}^{(4)} \\ \psi^{(2)} \end{pmatrix} = \underbrace{\begin{pmatrix} -\frac{1}{m}\left(2\zeta_2 B\,\dot{\boldsymbol{\Theta}} + \zeta_1 \dot{B}\,\dot{\boldsymbol{\Theta}}\right) \\ 0 \end{pmatrix}}_{I(\boldsymbol{\xi},\boldsymbol{\zeta})} + \underbrace{\begin{pmatrix} -\frac{1}{m}\mathbf{b} & -\frac{\zeta_1}{m} B J^{-1} \\ 0 & 0 \quad 0 \quad \frac{1}{I_z} \end{pmatrix}}_{D(\boldsymbol{\xi},\boldsymbol{\zeta})} \begin{pmatrix} \ddot{T} \\ \tau_\varphi \\ \tau_\vartheta \\ \tau_\psi \end{pmatrix} \tag{2.99}$$

In order to have the inputs appear in a non-singular way, D must be non-singular. Its determinant is

$$\det D = -\frac{T^2 \cos\varphi}{m^3 I_x I_y I_z} \tag{2.100}$$

hence D is non-singular if and only if $T \neq 0$ ($T = 0$ corresponds to free fall) and $\varphi \neq \pm\frac{\pi}{2}$. Away from the singularities, the sum of the relative degrees is

$$\sum_i r_i = 4 + 4 + 4 + 2 = 14 = n + \nu \tag{2.101}$$

hence *full state linearization* with no zero dynamics is achieved. The new state $\mathbf{z}$ that we choose through a diffeomorphism ϕ is given by the outputs and their derivatives:

$$\mathbf{z} = \phi(\boldsymbol{\xi},\boldsymbol{\zeta}) = \begin{pmatrix} \mathbf{p} & \dot{\mathbf{p}} & \ddot{\mathbf{p}} & \dddot{\mathbf{p}} & \psi & \dot{\psi} \end{pmatrix}^\top \in \mathbb{R}^{14} \tag{2.102}$$

we can act on $\mathbf{p}^{(4)}$ hence its derivatives of lower orders become part of the state, with:

$$\ddot{\mathbf{p}} = g\mathbf{e}_3 - \frac{\zeta_1}{m}\mathbf{b}(\boldsymbol{\Theta}) \tag{2.103}$$

$$\dddot{\mathbf{p}} = -\frac{1}{m}(\zeta_2 \mathbf{b}(\boldsymbol{\Theta}) + \zeta_1 B(\boldsymbol{\Theta})\dot{\boldsymbol{\Theta}}). \tag{2.104}$$

The linearizing and decoupling control law is

$$\mathbf{u} = D^{-1}(\boldsymbol{\xi},\boldsymbol{\zeta})(-I(\boldsymbol{\xi},\boldsymbol{\zeta}) + \mathbf{v}). \tag{2.105}$$

the closed loop system becomes:

$$x^{(4)} = v_1 \tag{2.106}$$

$$y^{(4)} = v_2 \tag{2.107}$$

$$z^{(4)} = v_3 \tag{2.108}$$

$$\ddot{\psi} = v_4. \tag{2.109}$$

The system is linear in the coordinates $\mathbf{z}$. To track a trajectory $\boldsymbol{\eta}_d$ we define the error $e_i = \eta_{id} - \eta_i$ for each i and set a PD (with higher order derivatives):

$$v_i = \eta_{id}^{(4)} + \sum_{j=1}^{4} c_{i,j-1} e_i^{(j-1)} \qquad i = 1,2,3 \tag{2.110}$$

$$v_4 = \ddot{\eta}_{4d} + c_{4,0}e_4 + c_{4,1}\dot{e}_4. \tag{2.111}$$

The system is stable and the tracking error, with an appropriate choice of the gains $c_{i,j}$, converges exponentially. The reference $\boldsymbol{\eta}_d$ must be differentiable four times at least for the first three components (position) and at least two times for the yaw ψ. The dynamic compensator has the following time evolution:

$$\dot{\zeta}_1 = \zeta_2 \tag{2.112}$$

$$\dot{\zeta}_2 = D_1^{-1}(\boldsymbol{\xi},\boldsymbol{\zeta})(-I(\boldsymbol{\xi},\boldsymbol{\zeta}) + \mathbf{v}) \tag{2.113}$$

$$T = \zeta_1 \tag{2.114}$$

$$\tau_\varphi = D_2^{-1}(\boldsymbol{\xi},\boldsymbol{\zeta})(-I(\boldsymbol{\xi},\boldsymbol{\zeta}) + \mathbf{v}) \tag{2.115}$$

$$\tau_\vartheta = D_3^{-1}(\boldsymbol{\xi},\boldsymbol{\zeta})(-I(\boldsymbol{\xi},\boldsymbol{\zeta}) + \mathbf{v}) \tag{2.116}$$

$$\tau_\psi = D_4^{-1}(\boldsymbol{\xi},\boldsymbol{\zeta})(-I(\boldsymbol{\xi},\boldsymbol{\zeta}) + \mathbf{v}) \tag{2.117}$$

with D_i^{-1} being the i-th row of D^{-1}, hence $D_i^{-1}(\xi,\zeta)(-I(\xi,\zeta)+\mathbf{v})$ is the scalar result of a product between row and column.

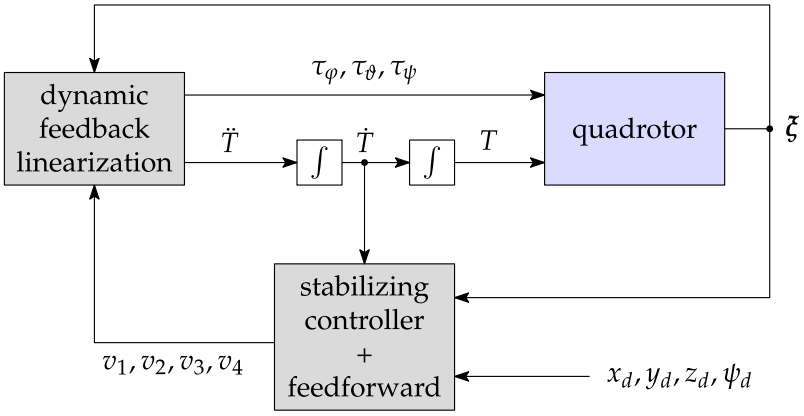


Figure 2.3: Block diagram of the control system.

2.2.4 Considering the Complete Model

In this section we discuss the dynamic feedback linearization for the quadrotor model, by removing the assumption of small angles. We neglect only the aerodynamics and gyroscopic effects:

$$\dot{x} = v_x \qquad \dot{\varphi} = p + s_\varphi \tan\vartheta q + c_\varphi \tan\vartheta r \tag{2.118}$$

$$\dot{y} = v_y \qquad \dot{\vartheta} = c_\varphi q - s_\varphi r \tag{2.119}$$

$$\dot{z} = v_z \qquad \dot{\psi} = s_\varphi \sec\vartheta q + c_\varphi \sec\vartheta r \tag{2.120}$$

$$\dot{v}_x = -(c_\psi s_\vartheta c_\varphi + s_\psi s_\varphi)\frac{T}{m} \qquad \dot{p} = \frac{I_y - I_z}{I_x} qr + \frac{\tau_\varphi}{I_x} \tag{2.121}$$

$$\dot{v}_y = -(s_\psi s_\vartheta c_\varphi - s_\varphi c_\psi)\frac{T}{m} \qquad \dot{q} = \frac{I_z - I_x}{I_y} pr + \frac{\tau_\vartheta}{I_y} \tag{2.122}$$

$$\dot{v}_z = g - c_\vartheta c_\varphi \frac{T}{m} \qquad \dot{r} = \frac{I_x - I_y}{I_z} pq + \frac{\tau_\psi}{I_z} \tag{2.123}$$

we are using the short notation for sin and cos. We denote $\mathbf{\Omega} = (p,q,r)^\top$ the angular velocity, the model in compact form can be rewritten as:

$$\ddot{\mathbf{p}} = g\mathbf{e}_3 - \frac{T}{m} R\mathbf{e}_3 \tag{2.124}$$

$$\dot{R} = RS(\mathbf{\Omega}) \tag{2.125}$$

$$J\dot{\mathbf{\Omega}} = -\mathbf{\Omega} \times J\mathbf{\Omega} + \mathbf{\Gamma}. \tag{2.126}$$

We compute the dynamic feedback linearization (DFL) by setting the same output and extension:

$$\boldsymbol{\eta} = \begin{pmatrix} x & y & z & \psi \end{pmatrix}^\top \tag{2.127}$$

$$\boldsymbol{\zeta} = \begin{pmatrix} T & \dot{T} \end{pmatrix}^\top \tag{2.128}$$

let's write out $\mathbf{p}^{(4)}$:

$$\mathbf{p}^{(3)} = \frac{d}{dt}\left(g\mathbf{e}_3 - \frac{T}{m} R\mathbf{e}_3\right) \tag{2.129}$$

$$= -\frac{T}{m}\frac{d}{dt}(R\mathbf{e}_3) - \frac{\dot{T}}{m} R\mathbf{e}_3 \tag{2.130}$$

$$= -\frac{1}{m} R(\dot{T}\mathbf{e}_3 + T\mathbf{\Omega} \times \mathbf{e}_3) \implies \tag{2.131}$$

$$\mathbf{p}^{(4)} = -\frac{1}{m} R \begin{pmatrix} T\dot{q} + 2\dot{T}q + Tpr \\ -T\dot{p} - 2\dot{T}p + Tqr \\ \ddot{T} - T(p^2+q^2) \end{pmatrix} \tag{2.132}$$

the snap $\mathbf{p}^{(4)}$ depends on $\dot{p}, \dot{q}, \ddot{T}$. We substitute $\dot{q}, \dot{p}$ as in Equations (2.121) and (2.122):

$$\mathbf{p}^{(4)} = -\frac{1}{m} R \begin{pmatrix} T(\frac{I_z - I_x}{I_y} pr + \frac{\tau_\vartheta}{I_y}) + 2\dot{T}q + Tpr \\ -T(\frac{I_y - I_z}{I_x} qr + \frac{\tau_\varphi}{I_x}) - 2\dot{T}p + Tqr \\ \ddot{T} - T(p^2 + q^2) \end{pmatrix} \tag{2.133}$$

In order to reorganize the snap equation, we define the following matrix and vector:

$$N = \begin{pmatrix} 0 & 0 & \frac{T}{I_y} \\ 0 & -\frac{T}{I_x} & 0 \\ 1 & 0 & 0 \end{pmatrix}, \quad \mathbf{n} = \begin{pmatrix} (1 + \frac{I_z - I_x}{I_y}) Tpr + 2\dot{T}q \\ (1 - \frac{I_y - I_z}{I_x}) Tqr - 2\dot{T}p \\ -T(p^2 + q^2) \end{pmatrix} \tag{2.134}$$

Equation (2.133) becomes:

$$\mathbf{p}^{(4)} = \underbrace{-\frac{1}{m} R\mathbf{n}}_{I_p(\xi,\zeta)} + \underbrace{-\frac{1}{m} RN}_{D_p(\xi,\zeta)} \begin{pmatrix} \ddot{T} \\ \tau_\varphi \\ \tau_\vartheta \end{pmatrix} \tag{2.135}$$

I_p, D_p are the translational part of the matrices I, D of the model in the form in Equation (2.81). Notice how the snap $\mathbf{p}^{(4)}$ does not depend on τ_ψ. In the simplified model the yaw dynamics $\ddot{\psi}$ was a simple double integrator, as seen in Equation (2.59). This does not hold anymore, since:

$$\dot{\psi} = s_\varphi \sec\vartheta q + c_\varphi \sec\vartheta r \tag{2.136}$$

before deriving $\ddot{\psi}$ we introduce the following scalar:

$$I_\psi = \frac{s_\varphi \frac{I_z - I_x}{I_y} pr + c_\varphi \frac{I_x - I_y}{I_z} pq}{c_\vartheta} + \frac{(c_\varphi q - s_\varphi r)\dot{\varphi}}{c_\vartheta} + \frac{(s_\varphi q + c_\varphi r)\tan\vartheta\,\dot{\vartheta}}{c_\vartheta} \tag{2.137}$$

the yaw acceleration is exactly:

$$\ddot{\psi} = I_\psi + \frac{s_\varphi}{c_\vartheta I_y}\tau_\vartheta + \frac{c_\varphi}{c_\vartheta I_z}\tau_\psi \tag{2.138}$$

now both the control torques $\tau_\vartheta, \tau_\psi$ act on $\ddot{\psi}$. The complete model is:

$$\begin{pmatrix} \mathbf{p}^{(4)} \\ \ddot{\psi} \end{pmatrix} = \underbrace{\begin{pmatrix} -\frac{1}{m} R\mathbf{n} \\ I_\psi \end{pmatrix}}_{I(\xi,\zeta)} + \underbrace{\begin{pmatrix} & -\frac{1}{m} RN & & 0 \\ & & & 0 \\ & & & 0 \\ 0 & 0 & \frac{s_\varphi}{c_\vartheta I_y} & \frac{c_\varphi}{c_\vartheta I_z} \end{pmatrix}}_{D(\xi,\zeta)} \begin{pmatrix} \ddot{T} \\ \tau_\varphi \\ \tau_\vartheta \\ \tau_\psi \end{pmatrix} \tag{2.139}$$

with

$$\det D = -\frac{T^2 \cos\varphi}{m^3 I_x I_y I_z \cos\vartheta} \tag{2.140}$$

hence the model has a singularity on $T = 0$ and $\varphi = \pm\frac{\pi}{2}$. Now we want to linearize this model, our virtual input is:

$$\mathbf{v} = \begin{pmatrix} v_1 & v_2 & v_3 & v_4 \end{pmatrix}^\top \tag{2.141}$$

with v_1, v_2, v_3 being relative to the snap and v_4 to the yaw. In order to linearize the snap we define the following vector:

$$\boldsymbol{\mu} = \begin{pmatrix} \mu_1 \\ \mu_2 \\ \mu_3 \end{pmatrix} = -mR^\top \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} - \mathbf{n} \tag{2.142}$$

The linearizing control law is

$$\ddot{T} = \mu_3 \tag{2.143}$$

$$\tau_\varphi = -\frac{I_x}{T}\mu_2 \tag{2.144}$$

$$\tau_\vartheta = \frac{I_y}{T}\mu_1 \tag{2.145}$$

$$\tau_\psi = \frac{I_z c_\vartheta}{c_\varphi}(v_4 - I_\psi) - \frac{I_z}{I_y}\tan\varphi\tau_\vartheta \tag{2.146}$$

the system becomes:

$$x^{(4)} = v_1 \tag{2.147}$$
$$y^{(4)} = v_2 \tag{2.148}$$
$$z^{(4)} = v_3 \tag{2.149}$$
$$\ddot{\psi} = v_4. \tag{2.150}$$

The following table compares the simplified model with the complete one:

	approximate	**complete**
rotational kinematics	$\dot{\boldsymbol{\Theta}} \simeq \boldsymbol{\Omega}$	$\dot{\boldsymbol{\Theta}} = M(\boldsymbol{\Theta})^{-1}\boldsymbol{\Omega}$
rotational dynamics	$\ddot{\boldsymbol{\Theta}} = J^{-1}\boldsymbol{\Gamma}$	$J\dot{\boldsymbol{\Omega}} = -\boldsymbol{\Omega} \times J\boldsymbol{\Omega} + \boldsymbol{\Gamma}$
jerk $\mathbf{p}^{(3)}$	$-\frac{1}{m}(\dot{T}\mathbf{b} + TB\dot{\boldsymbol{\Theta}})$	$-\frac{1}{m}R(Tq, -Tp, \dot{T})^\top$
position block of D	$-\frac{1}{m}(\mathbf{b} \quad TBJ^{-1})$	$-\frac{1}{m}RN(T)$
yaw row of D	$(0, 0, 0, \frac{1}{I_z})$	$\left(0, 0, \frac{s_\varphi}{c_\vartheta I_y}, \frac{c_\varphi}{c_\vartheta I_z}\right)$
det D	$-\dfrac{T^2 c_\varphi}{m^3 I_x I_y I_z}$	$-\dfrac{T^2 c_\varphi}{m^3 I_x I_y I_z c_\vartheta}$

2.2.5 Flatness

Definition 2.2.2 : Differentially Independent

Let $\mathbf{y}$ be a vector of quantities that changes over time. The components of $\mathbf{y}$ are **differentially independent** if there is no differential equation that links the individual components of $\mathbf{y}$ together, i.e. it is not possible to find an equation of the form:

$$\Phi(\mathbf{y}, \dot{\mathbf{y}}, \ddot{\mathbf{y}}, \dots) = \mathbf{0}. \tag{2.151}$$

Definition 2.2.3 : Differentially Flat System

A system $\dot{\boldsymbol{\xi}} = f(\boldsymbol{\xi}, \mathbf{u})$ with $\mathbf{u} \in \mathbb{R}^m$ is **differentially flat** if there exist m outputs:

$$\mathbf{y} = h(\boldsymbol{\xi}, \mathbf{u}, \dot{\mathbf{u}}, \dots, \mathbf{u}^{(k)}) \tag{2.152}$$

called **flat outputs** such that the state $\boldsymbol{\xi}$ and the inputs $\mathbf{u}$ can be rewritten as functions of $\mathbf{y}$ and its derivatives (up to a finite order):

$$\boldsymbol{\xi} = \Psi_{\xi}(\mathbf{y}, \dot{\mathbf{y}}, \dots, \mathbf{y}^{(r_\xi)}) \tag{2.153}$$
$$\mathbf{u} = \Psi_{u}(\mathbf{y}, \dot{\mathbf{y}}, \dots, \mathbf{y}^{(r_u)}) \tag{2.154}$$

and the components of $\mathbf{y}$ are differentially independent.

If a system is differentially flat, the trajectories of the system are in one-to-one correspondence with arbitrary smooth curves $\mathbf{y}(t)$: there are no differential equations to integrate.

Observation 2.2.1

There does not exist a general test to check if a system is differentially flat, flat outputs are found by inspection, often with a physical meaning.

The quadrotor model is one of the best-known examples of a system with flat outputs, which are exactly:

$$\mathbf{y} = \begin{pmatrix} x & y & z & \psi \end{pmatrix}^\top \tag{2.155}$$

by considering the inputs

$$\mathbf{u} = \begin{pmatrix} T & \tau_\varphi & \tau_\vartheta & \tau_\psi \end{pmatrix}^\top \tag{2.156}$$

we want to derive the map Ψ such that:

$$\mathbf{u} = \Psi(\mathbf{y}, \dot{\mathbf{y}}, \dots, \mathbf{y}^{(r)}) \tag{2.157}$$

first, we express T in function of $\ddot{\mathbf{p}}$, by the dynamic model:

$$\ddot{\mathbf{p}} = g\mathbf{e}_3 - \frac{T}{m} R\mathbf{e}_3 \implies \tag{2.158}$$

$$T = m\|g\mathbf{e}_3 - \ddot{\mathbf{p}}\| \tag{2.159}$$

the versor of the thrust direction is:

$$\mathbf{b} = \frac{g\mathbf{e}_3 - \ddot{\mathbf{p}}}{\|g\mathbf{e}_3 - \ddot{\mathbf{p}}\|} \tag{2.160}$$

from the jerk equation we can define a new vector:

$$\mathbf{w} = -mR^\top \mathbf{p}^{(3)} \tag{2.161}$$

and get the following identities:

$$q = \frac{w_1}{T} \tag{2.162}$$

$$p = -\frac{w_2}{T} \tag{2.163}$$

$$\dot{T} = w_3 \tag{2.164}$$

and

$$r = \frac{c_\vartheta \dot{\psi} - s_\varphi q}{c_\varphi} \tag{2.165}$$

from the snap equation, we define:

$$\boldsymbol{\mu} = -mR^\top \mathbf{p}^{(4)} - \mathbf{n} \tag{2.166}$$

and we can write:

$$\ddot{T} = \mu_3 \tag{2.167}$$

$$\tau_\vartheta = \frac{I_y}{T}\mu_1 \tag{2.168}$$

$$\tau_\varphi = -\frac{I_x}{T}\mu_2 \tag{2.169}$$

$$\tau_\psi = \frac{I_z c_\vartheta}{c_\varphi}(\ddot{\psi} - I_\psi) - \frac{I_z}{I_y}\tan\varphi\tau_\vartheta \tag{2.170}$$

now that we have defined the map $\mathbf{u} = \Psi(\mathbf{y}, \dot{\mathbf{y}}, \dots, \mathbf{y}^{(4)})$, we can define any trajectory at least four times differentiable $\mathbf{y}(t)$ and get immediately through Ψ the required inputs to execute $\mathbf{y}(t)$.

2.3 Geometric Control on $SO(3)$

The state space of the rotational dynamics is $\mathbb{R}^3 \times SO(3)$, this is a **topological obstruction**: due to this, it is impossible with a continuous feedback law to make an equilibrium point *globally asymptotically stable*. The best thing that we can do is to use the information about the geometry of the state space and define an error on that. By doing so, the best achievable result with a continuous feedback law is **almost global convergence** (a desired equilibrium attracts trajectories starting from almost all initial conditions,

except a zero-measure set). The **geometric controller** is designed by using directly $SO(3)$ to represent the desired attitude of the UAV.

We recall the quadrotor dynamics:

$$\ddot{\mathbf{p}} = g\mathbf{e}_3 - \frac{T}{m}R\mathbf{e}_3 \tag{2.171}$$

$$\dot{R} = RS(\boldsymbol{\Omega}) \tag{2.172}$$

$$J\dot{\boldsymbol{\Omega}} + \boldsymbol{\Omega} \times J\boldsymbol{\Omega} = \boldsymbol{\Gamma} \tag{2.173}$$

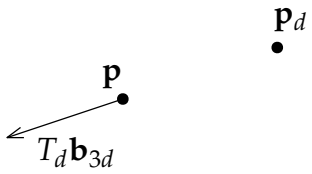
with $\boldsymbol{\Gamma}, T$ being the control inputs. We define a new operator $^\vee$: this is the inverse of the operator that assigns to a vector its associated skew-symmetric matrix:

$$A = S(\mathbf{a}) \tag{2.174}$$

$$A^\vee = (S(\mathbf{a}))^\vee = \mathbf{a} \tag{2.175}$$

The controller is divided in two loops, the outer loop for the position, the inner loop for the attitude. We denote $\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3$ the unit vector columns of R (these vectors are the one denoted $\mathbf{x}_b, \mathbf{y}_b, \mathbf{z}_b$ in Figure 1.3). To impose a desired attitude, we define a desired rotation matrix R_d, with column vectors $\mathbf{b}_{1d}, \mathbf{b}_{2d}, \mathbf{b}_{3d}$. The two loops work as follows:

- The trajectory tracking controller takes in input a desired Cartesian position $\mathbf{p}_d$, and produces in output the required thrust intensity T and thrust direction $\mathbf{b}_{3d}$.



- $\mathbf{b}_{3d}$, and $\mathbf{b}_{1d}$ (desired yaw) are used to define R_d, that will be used by the attitude tracking controller to compute the required torques $\boldsymbol{\Gamma}$.
- $\boldsymbol{\Gamma}$ and T are the computed control inputs for the quadrotor dynamics.

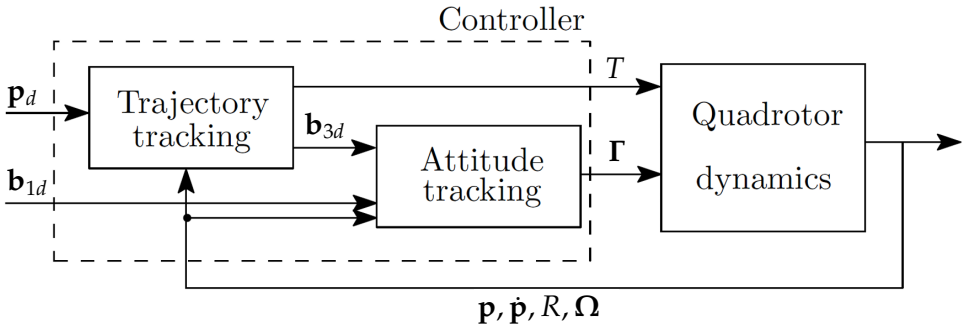


Figure 2.4: Block diagram of the geometric controller.

First of all we define the Cartesian error for the position:

$$\mathbf{e}_p = \mathbf{p} - \mathbf{p}_d \tag{2.176}$$

$$\mathbf{e}_v = \dot{\mathbf{p}} - \dot{\mathbf{p}}_d \tag{2.177}$$

the resulting positional control action $\mathbf{a}$ is built with a PD law + gravity compensation + feedforward:

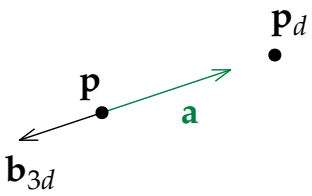
$$\mathbf{a} = -k_p\mathbf{e}_p - k_d\mathbf{e}_v - mg\mathbf{e}_3 + m\ddot{\mathbf{p}}_d. \tag{2.178}$$

The thrust force that we can impose is $-TR\mathbf{e}_3$, and we would like to make this equal to the positional control action:

$$-TR\mathbf{e}_3 = \mathbf{a} \tag{2.179}$$

we cannot assign directly $-TR\mathbf{e}_3$, but we can assign T and the control torques to control R. The thrust direction desired is the following:

$$\mathbf{b}_{3d} = -\frac{\mathbf{a}}{\|\mathbf{a}\|}. \tag{2.180}$$



This requires $\mathbf{a} \neq \mathbf{0}$ (no free fall). The thrust intensity is:

$$T = \|\mathbf{a}\|\mathbf{b}_{3d}^\top \mathbf{b}_3 \tag{2.181}$$

so T is the thrust intensity to actuate, now we have to describe how the inner attitude controller will achieve the desired attitude R_d defined by the desired thrust direction $\mathbf{b}_{3d}$. The attitude is constructed as follows:

- Let $\mathbf{b}_{1d}$ be the desired yaw. We build the vector:

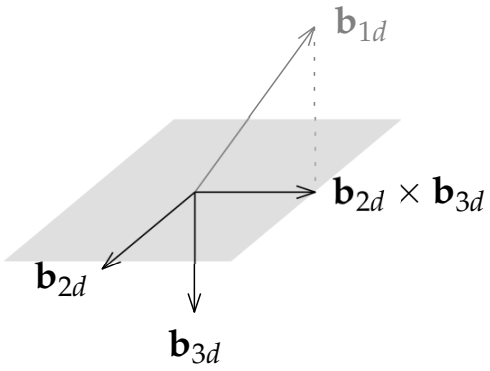
$$\mathbf{b}_{2d} = \frac{\mathbf{b}_{3d} \times \mathbf{b}_{1d}}{\|\mathbf{b}_{3d} \times \mathbf{b}_{1d}\|} \tag{2.182}$$

- $\mathbf{b}_{2d}$ and $\mathbf{b}_{3d}$ will be completed to an orthonormal basis by their cross product:

$$\mathbf{b}_{2d} \times \mathbf{b}_{3d} \tag{2.183}$$

  by this construction, $\mathbf{b}_{2d} \times \mathbf{b}_{3d}$ will be the projection of $\mathbf{b}_{1d}$ on the plane orthogonal to $\mathbf{b}_{3d}$. Ultimately:

$$R_d = \begin{pmatrix} \mathbf{b}_{2d} \times \mathbf{b}_{3d} & \mathbf{b}_{2d} & \mathbf{b}_{3d} \end{pmatrix}. \tag{2.184}$$



This construction is well defined if $\mathbf{b}_{1d}$ is not parallel to $\mathbf{b}_{3d}$. Now we have to define an error between matrices in $SO(3)$, so we define the error function $\Psi : SO(3) \times SO(3) \to [0, 2]$ as follows:

$$\Psi(R, R_d) = \frac{1}{2}\text{tr}\left[I - R_d^\top R\right] \tag{2.185}$$

this function has some important properties:

- $R_d^\top R$ is the relative rotation between the current and desired attitude.

- Let $R_d^\top R$ be the rotation about $\mathbf{r}$ of θ radians (axis-angle representation). Then:

$$\text{tr}\left[R_d^\top R\right] = 1 + 2\cos\theta \tag{2.186}$$

hence:

$$\Psi(R, R_d) = 1 - \cos\theta \in [0, 2]. \tag{2.187}$$

- The error is zero if the matrices are identical:

$$\Psi(R, R_d) = 0 \iff R = R_d. \tag{2.188}$$

- The error is less than 1 if and only if the angle θ is less than $\frac{\pi}{2}$:

$$\Psi(R, R_d) < 1 \iff \theta < \frac{\pi}{2}. \tag{2.189}$$

- The error is maximum when the angle θ is exactly π:

$$\Psi(R, R_d) = 2 \iff \theta = \pi. \tag{2.190}$$

Proposition 2.3.1 : Almost Global Convergence

At the beginning of this section we have said that the geometric controller achieves almost global convergence. This means that asymptotic convergence is **guaranteed** if the error defined by Ψ between the current attitude R and desired attitude R_d is less than 2.

At each attitude R, asymptotic convergence is guaranteed on the subset:

$$L_2 = \{R_d \in SO(3) \text{ such that } \Psi(R, R_d) < 2\} \subset SO(3). \tag{2.191}$$

L_2 almost covers $SO(3)$, meaning that the set $SO(3) \setminus L_2$ has zero measure.

Now that we defined a desired attitude R_d and an error $\Psi(R, R_d)$ that is a measure of how distant we are from the desired configuration, we have to define how we choose the control torques $\boldsymbol{\Gamma}$. First of all we consider a *small perturbation* on R, let $\boldsymbol{\eta}$ be a vector of infinitesimal rotation, the perturbation is:

$$\delta R = RS(\boldsymbol{\eta}) \tag{2.192}$$

now we compute how Ψ changes when R changes (and R_d is constant). Since the trace operator is linear, the variation is pushed inside the operator:

$$\delta\Psi = \delta\frac{1}{2}\text{tr}\left[I - R_d^\top R\right] \tag{2.193}$$

$$= \frac{1}{2}\text{tr}\left[\delta I - \delta(R_d^\top R)\right] \tag{2.194}$$

$$= \frac{1}{2}\text{tr}\left[\delta I - (R_d^\top \delta R)\right] \tag{2.195}$$

I is constant hence δI is the zero matrix, then:

$$\delta\Psi = -\frac{1}{2}\text{tr}\left[R_d^\top RS(\boldsymbol{\eta})\right] \tag{2.196}$$

from a series of algebraic steps (that we omit), the variation can be rewritten as follows:

$$\delta\Psi = \underbrace{\frac{1}{2}\cdot\left(R_d^\top R - R^\top R_d\right)^\vee}_{\mathbf{e}_R}\cdot\boldsymbol{\eta} \tag{2.197}$$

$$= \mathbf{e}_R^\top\boldsymbol{\eta} \tag{2.198}$$

we have found the vector $\mathbf{e}_R$ that is the **gradient** of Ψ, represents the direction (expressed in the body frame) along which the error function Ψ grows most rapidly. It can be proved that, if $\mathbf{r}, \theta$ is the axis-angle representation of the relative rotation $R_d^\top R$, then the gradient can be written as:

$$\mathbf{e}_R = \sin\theta \mathbf{r} \tag{2.199}$$

furthermore its squared magnitude is:

$$\|\mathbf{e}_R\|^2 = \underbrace{\sin^2\theta \mathbf{r}^\top \mathbf{r} = \sin^2\theta}_{\mathbf{r} \text{ is a unit vector}} = \Psi(2-\Psi). \tag{2.200}$$

Let $\psi_2 < 2$ be a constant value decided a priori by design, it holds that, for any attitude R such that $\Psi(R, R_d) \leq \psi_2$, the following bounds hold:

$$\frac{1}{2}\|\mathbf{e}_R\|^2 \leq \Psi \leq \frac{1}{2-\psi_2}\|\mathbf{e}_R\|^2. \tag{2.201}$$

- From the lower bound ($\frac{1}{2}\|\mathbf{e}_R\|^2 \leq \Psi$) we deduce that if the configuration error Ψ tends to zero, then the error vector $\mathbf{e}_R$ must also tend to zero ($\Psi \to 0 \implies \|\mathbf{e}_R\| \to 0$).
- From the upper bound ($\Psi \leq \frac{1}{2-\psi_2}\|\mathbf{e}_R\|^2$) we deduce the vice versa: within the safe region $\Psi \leq \psi_2 < 2$, if the error vector $\mathbf{e}_R$ vanishes, then Ψ must also vanish, i.e. $R = R_d$.

Observation 2.3.1

The bounds on Equation (2.201) show that Ψ is bounded between two quadratic forms of the attitude error vector $\mathbf{e}_R$; therefore, within this sublevel set, $\Psi(R, R_d)$ is positive definite with respect to $\mathbf{e}_R$ (i.e., $\Psi = 0 \iff \mathbf{e}_R = \mathbf{0}$).

After defining the attitude error, we define the error on angular velocity; let $\mathbf{\Omega}_d = (p_d, q_d, r_d)$ be the desired angular velocity, the error $\mathbf{e}_\Omega$ is:

$$\mathbf{e}_\Omega = \mathbf{\Omega} - R^\top R_d \mathbf{\Omega}_d. \tag{2.202}$$

$\mathbf{\Omega}$ is expressed in the current body frame defined by R while $\mathbf{\Omega}_d$ is expressed in the desired body frame defined by R_d, hence $R_d\mathbf{\Omega}_d$ is the desired angular velocity in the base frame (also called inertial frame). $\mathbf{e}_\Omega$ is exactly the angular velocity vector that governs the time derivative of the relative rotation matrix $R_d^\top R$:

$$\begin{aligned}
\frac{d}{dt}(R_d^\top R) &= \dot{R}_d^\top R + R_d^\top \dot{R} && (2.203)\\
&= -S(\mathbf{\Omega}_d)R_d^\top R + R_d^\top R S(\mathbf{\Omega}) && (2.204)\\
&= R_d^\top R(S(\mathbf{\Omega}) - R^\top R_d S(\mathbf{\Omega}_d) R_d^\top R) && (2.205)\\
&= R_d^\top R\, S(\mathbf{\Omega} - R^\top R_d \mathbf{\Omega}_d) && (2.206)\\
&= R_d^\top R\, S(\mathbf{e}_\Omega). && (2.207)
\end{aligned}$$

It can be proved that the derivative of the attitude error function is the dot product between the gradient and the angular velocity error:

$$\dot{\Psi} = -\frac{1}{2}\text{tr}\left[\frac{d}{dt}(R_d^\top R)\right] = -\frac{1}{2}\text{tr}\left[R_d^\top R S(\mathbf{e}_\Omega)\right] = \frac{1}{2}\mathbf{e}_\Omega^\top (R_d^\top R - R^\top R_d)^\vee = \mathbf{e}_R^\top \mathbf{e}_\Omega. \tag{2.208}$$

Let's recap what we have done until now:

1. We defined the inner-outer loop structure of the geometric controller.
2. We have seen how the outer loop (trajectory) generates the desired thrust direction for the inner loop, defined by $\mathbf{b}_{3d}$.

3. The inner loop constructs the matrix R_d, and defines an error Ψ between the current attitude R and the desired one R_d.
4. We have defined the errors $\mathbf{e}_R = \nabla\Psi$ and $\mathbf{e}_\Omega$, and we have seen how $\dot{\Psi} = \mathbf{e}_R^\top \mathbf{e}_\Omega$.

What we have to do now?: We derive the dynamics of the errors, and choose the control torques $\boldsymbol{\Gamma}$ in order to achieve a closed loop system that asymptotically nullifies the errors.

The dynamics of the attitude error is the following:

$$\dot{\mathbf{e}}_R = \frac{1}{2}\left(R_d^\top R S(\mathbf{e}_\Omega) + S(\mathbf{e}_\Omega) R^\top R_d\right)^\vee \tag{2.209}$$

$$= \underbrace{\frac{1}{2}\left(\text{tr}\left[R^\top R_d\right] I - R^\top R_d\right)}_{C(R_d^\top R)} \mathbf{e}_\Omega \tag{2.210}$$

The matrix $\frac{1}{2}\left(\text{tr}\left[R^\top R_d\right] I - R^\top R_d\right)$ can be rewritten as $C(R_d^\top R)$ with C being a matrix with norm less or equal than 1:

$$\dot{\mathbf{e}}_R = C(R_d^\top R)\mathbf{e}_\Omega, \qquad \|C\| \leq 1. \tag{2.211}$$

So the velocity of the attitude error depends on the angular velocity error, its dynamics is:

$$\dot{\mathbf{e}}_\Omega = \dot{\boldsymbol{\Omega}} - \frac{d}{dt}(R^\top R_d \boldsymbol{\Omega}_d) \tag{2.212}$$

$$= \dot{\boldsymbol{\Omega}} + S(\boldsymbol{\Omega}) R^\top R_d \boldsymbol{\Omega}_d - R^\top R_d \underbrace{S(\boldsymbol{\Omega}_d)\boldsymbol{\Omega}_d}_{\mathbf{0}} - R^\top R_d \dot{\boldsymbol{\Omega}}_d \tag{2.213}$$

$$= \dot{\boldsymbol{\Omega}} + S(\boldsymbol{\Omega}) R^\top R_d \boldsymbol{\Omega}_d - R^\top R_d \dot{\boldsymbol{\Omega}}_d \tag{2.214}$$

Let's multiply both sides by the quadrotor inertia matrix J:

$$J\dot{\mathbf{e}}_\Omega = J\dot{\boldsymbol{\Omega}} + J\left(S(\boldsymbol{\Omega}) R^\top R_d \boldsymbol{\Omega}_d - R^\top R_d \dot{\boldsymbol{\Omega}}_d\right) \tag{2.215}$$

we substitute $J\dot{\boldsymbol{\Omega}}$ with the dynamics in Equation (2.173):

$$J\dot{\mathbf{e}}_\Omega = \boldsymbol{\Gamma} - \boldsymbol{\Omega} \times J\boldsymbol{\Omega} + J\left(S(\boldsymbol{\Omega}) R^\top R_d \boldsymbol{\Omega}_d - R^\top R_d \dot{\boldsymbol{\Omega}}_d\right). \tag{2.216}$$

The control input that we choose to stabilize the system is the following:

$$\boldsymbol{\Gamma} = -k_R \mathbf{e}_R - k_\Omega \mathbf{e}_\Omega + \boldsymbol{\Omega} \times J\boldsymbol{\Omega} - J(S(\boldsymbol{\Omega}) R^\top R_d \boldsymbol{\Omega}_d - R^\top R_d \dot{\boldsymbol{\Omega}}_d) \tag{2.217}$$

let's break it down:

- $-k_R \mathbf{e}_R - k_\Omega \mathbf{e}_\Omega$ is a PD action on the errors in $SO(3)$,
- $\boldsymbol{\Omega} \times J\boldsymbol{\Omega}$ to cancel the gyroscopic terms,
- $J(S(\boldsymbol{\Omega}) R^\top R_d \boldsymbol{\Omega}_d - R^\top R_d \dot{\boldsymbol{\Omega}}_d)$ is a feedforward action of the desired angular acceleration.

The closed loop dynamics achieved with this choice of $\boldsymbol{\Gamma}$ is the following:

$$J\dot{\mathbf{e}}_\Omega = -k_R \mathbf{e}_R - k_\Omega \mathbf{e}_\Omega \tag{2.218}$$

$$\dot{\mathbf{e}}_R = C(R_d^\top R)\mathbf{e}_\Omega \tag{2.219}$$

$$\frac{d}{dt}(R_d^\top R) = R_d^\top R S(\mathbf{e}_\Omega) \tag{2.220}$$

TODO: slide 04 pag 19